

1. Table of contents

1. Rezumat	3
1.1. Stadiul actual	3
1.2. Fundamentare teoretică.....	4
1.3. Implementarea soluției adoptate.....	5
1.4. Rezultate experimentale.....	6
2. Work planning.....	8
3. State of the art	9
3.1. Project context	9
3.2. Project objectives.....	10
3.3. Problem statement.....	11
4. Theoretical Fundamentals.....	12
4.1. Foundations of deep learning	12
4.2. Advanced architectural components	13
4.2.1. Residual learning framework	13
4.2.2. Analysis of gradient flows.....	13
4.2.3. Bottleneck architecture theory.....	14
4.2.4. Feature pyramid networks	14
4.3. Object detection.....	15
4.3.1. Comparing one-stage and two-stage detection methods.....	15
4.3.2. Region proposal networks (RPN) theory	15
4.3.3. Non-Maximum suppression (NMS) algorithm.....	16
4.4. Mask R-CNN.....	16
4.4.1. R-CNN.....	17
4.5. Loss functions and optimization.....	17
4.5.1. Multi-Task learning	17
4.5.2. Classification loss	18
4.5.3. Mask loss.....	18
4.6. Optimization algorithms.....	19
4.6.1. COCO evaluation.....	19
4.6.2. Mathematical formulation of COCO evaluation metrics.....	20

5. Implementation	22
5.1. Data usage	22
5.2. Model	24
5.2.1. Mask R-CNN	24
5.2.2. ResNet-50	25
5.2.3. Integrated system architecture	27
5.3. Training.....	27
5.4. Evaluation	29
5.5. Model comparison	31
6. Experimental results	33
6.1. Dataset definition.....	33
6.1.1. Dataset selection	33
6.1.2. Label mapping.....	35
6.2. Optimization	36
6.3. Parameters sets comparison	36
6.3.1. Set results comparison.....	38
6.4. Subsets comparison results.....	41
6.5. Visual evaluation	45
7. Conclusion.....	48
Appendix	49
References :.....	62

1. Rezumat

1.1. Stadiul actual

În ultimele două decenii, evoluția accelerată a tehnologiilor digitale și a infrastructurilor de calcul a condus la apariția unor forme de conținut media care depășesc granițele tradiționale dintre autentic și artificial. Dintre acestea, conținut falsificat digital reprezintă una dintre cele mai provocatoare manifestări, atât din punct de vedere tehnologic, cât și social. Acesta este cunoscut sub denumirea de „deepfake”. Termenul desemnează materiale vizuale sau audiovizuale generate cu ajutorul unor algoritmi de învățare profundă, în special rețele neuronale convoluționale (CNN) și rețele generative adversariale (GAN), capabile să înlocuiască fețe, să modifice expresii faciale sau să recreeze scene întregi cu un realism impresionant [1].

Dacă în fazele incipiente deepfake-urile erau relativ ușor de identificat datorită imperfecțiunilor vizibile, precum artefactele de textură, inconsecvențele cromatice sau sincronizarea slabă a expresiilor, în prezent acestea au atins un nivel de sofisticare care le face extrem de greu de distins de materialele originale. Modelele moderne sunt antrenate pe seturi masive de date și dispun de capacitatea de a reproduce micro-expresii, reflexii subtile ale luminii pe piele, variații dinamice ale mișcărilor faciale și chiar detalii de context, precum fundaluri complexe sau obiecte în mișcare [2].

Această perfecționare rapidă a tehnicii a generat preocupări majore în multiple domenii. În securitatea informațională, deepfake-urile pot fi utilizate pentru campanii de dezinformare, fraudă sau atacuri de inginerie socială. În criminalistica digitală, ele complică analiza probelor vizuale, necesitând instrumente sofisticate pentru autentificare. În mediul juridic, devine din ce în ce mai dificil să se stabilească validitatea unei dovezi video prezentate în instanță. Totodată, există implicații și în domeniul protecției datelor personale, deoarece oricine poate deveni victimă a unor manipulări vizuale fără consimțământ [3].

În paralel, trebuie menționat și potențialul pozitiv al acestor tehnologii. În cinematografie și divertisment, deepfake-ul permite recrearea actorilor decedați, îmbunătățirea efectelor speciale sau adaptarea conținutului pentru localizare culturală. În educație, poate fi utilizat pentru simulări și reconstituiri istorice, iar în medicină pentru antrenarea sistemelor de recunoaștere facială în detectarea unor patologii rare. Această dualitate – între utilitate și pericol – face ca fenomenul să fie extrem de controversat și să necesite reglementări, dar și instrumente tehnice de detecție cât mai robuste [3].

Pe măsură ce metodele tradiționale de analiză a imaginilor, bazate pe histogramă de culoare, descriptorii de textură (GLCM, LBP) sau măsurători geometrice, și-au demonstrat limitele, comunitatea științifică a migrat spre abordări avansate de învățare profundă. Acestea se bazează pe modele integrate de la intrare la ieșire, care învață automat caracteristici complexe din date brute și nu depind de selecția manuală a unor trăsături vizuale. Totodată, modelele moderne pot integra simultan sarcini de detecție, localizare și segmentare, ceea ce le conferă un avantaj major în analiza scenariilor complexe, unde pot apărea mai multe fețe într-un singur cadru sau unde calitatea imaginii este compromisă prin compresie ori iluminare deficitară.

În acest context, studiul de față se fundamentează pe utilizarea arhitecturilor de tip Mask R-CNN, care îmbină detecția obiectelor cu segmentarea la nivel de pixel, și pe exploatarea datasetului OpenForensics [4], unul dintre cele mai cuprinzătoare seturi publice disponibile pentru detecția și segmentarea conținutului deepfake. Spre deosebire de alte colecții de date, OpenForensics oferă imagini în condiții reale, nestructurate, capturate în condiții naturale, cu multiple variații de fundal, rezoluție și iluminare, și include atât exemple reale, cât și falsuri generate prin metode diverse,

precum face swap, reenactment sau deepfake clasic. În plus, structura sa în format COCO asigură compatibilitatea directă cu arhitecturile de ultimă generație și facilitează evaluarea standardizată a performanței [5] [4].

1.2. Fundamentare teoretică

Detectarea și segmentarea deepfake-urilor se sprijină pe un cadru metodologic complex, care presupune o serie de etape consecutive de la preprocesarea datelor brute, la extragerea automată a caracteristicilor, segmentarea precisă a zonelor alterate și până la evaluarea finală a performanței modelului. Această abordare integrată de la intrare la ieșire se distinge prin faptul că reduce dependența de tehnicile tradiționale bazate pe selectarea manuală a trăsăturilor și valorifică puterea rețelelor neuronale convoluționale adânci, capabile să învețe reprezentări ierarhice sofisticate direct din date[3].

Preprocesarea constituie punctul de intrare în pipeline-ul metodologic și are rolul de a standardiza datele, reducând variațiile irelevante dintre imagini. În mod tipic, acest pas include redimensionarea imaginilor la o rezoluție fixă, conversia acestora în tensori compatibili cu framework-urile de învățare profundă și normalizarea pixelilor conform valorilor medii și deviațiilor standard asociate dataseturilor mari de referință, precum ImageNet. În unele cazuri, se aplică și corecții suplimentare de contrast, luminozitate și balans de alb, menite să uniformizeze imaginile de intrare și să elimine influența factorilor externi, cum ar fi iluminarea neuniformă sau setările tehnice ale camerei[6] .

Odată pregătite datele, etapa esențială devine extragerea caracteristicilor, realizată de rețele neuronale convoluționale. Spre deosebire de descriptorii manuali, cum ar fi modelele de textură binară locală (Local Binary Patterns) sau matricea de co-ocurență a nivelurilor de gri (GLCM), rețele neuronale convoluționale (CNN) sunt capabile să construiască reprezentări ierarhice complexe: primele straturi învață să detecteze margini, texturi și contraste simple, în timp ce straturile adânci surprind structuri abstracte, cum ar fi forma globală a feței sau relațiile dintre diferite regiuni faciale. Această capacitate le permite să distingă detalii subtile care fac diferența dintre o imagine autentică și una alterată digital [7].

Segmentarea, aflată în centrul procesului de detectare, are rolul de a identifica la nivel de pixel zonele falsificate. Într-o imagine care conține o față alterată, segmentarea înseamnă generarea unei hărți binare în care fiecare pixel este clasificat drept autentic sau falsificat. Spre deosebire de detecția clasică, care se limitează la stabilirea unui bounding box în jurul regiunii suspecte, segmentarea oferă o granularitate suplimentară, esențială pentru aplicațiile criminalistice, unde precizia delimitării poate constitui o probă determinantă [8].

Arhitectura Mask R-CNN, aflată la baza acestui proiect, se înscrie în paradigma detectoarelor „doua-etape” și reprezintă o extensie naturală a Faster R-CNN. Ea este alcătuită din mai multe componente cheie. În primul rând, backbone-ul, care poate fi ResNet-50 sau ResNet-101, combinat cu Feature Pyramid Network (FPN), are rolul de a extrage hărți de caracteristici robuste din imagine, capabile să surprindă atât detalii locale, cât și informații de nivel global. În al doilea rând, Region Proposal Network (RPN) generează regiuni candidate susceptibile de a conține obiecte, în acest caz fețe sau fragmente de fețe. Apoi, modulul ROI Align aliniază precis aceste regiuni cu hărțile de trăsături, eliminând discrepanțele spațiale care apăreau în arhitecturile anterioare. În final, capul de detecție este divizat în două ramuri: una pentru clasificare și regresia bounding box-urilor, și una suplimentară, dedicată segmentării binare, care produce măști detaliate pentru fiecare obiect detectat [9] [10].

Funcția de pierdere totală a Mask R-CNN este compusă din trei termeni principali: pierderea de clasificare (cross-entropy), care penalizează erorile în atribuirea etichetelor „real” și „fake”; pierderea de regresie a bounding box-urilor (smooth L1), care asigură localizarea precisă a regiunilor; și pierderea de segmentare (binary cross-entropy), care optimizează calitatea măștilor binare. Această abordare de învățare multi-task asigură că modelul nu doar identifică și clasifică obiectele, ci și segmentează corect zonele alterate [10] [1].

Evaluarea performanței modelelor de segmentare și detecție este realizată conform standardului COCOeval, un set de metrici recunoscut la nivel internațional. Printre acestea, Intersection over Union (IoU) măsoară gradul de suprapunere dintre predicțiile modelului și adnotările reale, în timp ce mean Average Precision (mAP) calculează precizia medie la diverse praguri IoU, oferind o imagine nuanțată asupra performanței în condiții diferite. Recall-ul, la rândul său, evaluează sensibilitatea modelului, indicând cât de bine reușește acesta să detecteze toate cazurile pozitive. În plus, analiza class-wise, realizată separat pentru clasele „real” și „fake”, permite identificarea diferențelor de comportament și ajustarea fină a parametrilor în funcție de aceste observații [8].

Acest cadru teoretic oferă fundamentul pentru implementarea practică a soluției, în care Mask R-CNN este adaptată și antrenată pe datasetul OpenForensics, cu scopul de a atinge un echilibru optim între acuratețea detecției, precizia segmentării și robustețea generalizării la date reale și variate [10].

1.3. Implementarea soluției adoptate

Implementarea soluției propuse s-a concentrat pe utilizarea unei arhitecturi Mask R-CNN preantrenate, adaptată ulterior pentru sarcina specifică de detecție și segmentare a deepfake-urilor. Alegerea framework-ului PyTorch a fost naturală, datorită flexibilității sale, suportului pentru rețele complexe și comunității active, care facilitează rezolvarea problemelor tehnice. În plus, PyTorch permite o interacțiune directă cu modelul și oferă instrumente eficiente pentru antrenare, vizualizare și evaluare [10].

Primul pas a fost integrarea setului de date OpenForensics [4], ale cărui adnotări în format COCO au permis încărcarea și prelucrarea datelor prin clasele Dataset și DataLoader din PyTorch. Pentru a asigura rezultate experimentale corecte, datasetul a fost împărțit în trei subseturi: antrenare (70%), validare (15%) și testare (15%). Această împărțire echilibrată a permis optimizarea parametrilor și evaluarea performanței pe date complet noi pentru model [4].

În etapa de preprocesare, imaginile au fost redimensionate la o rezoluție standard de 224×224 pixeli, astfel încât să existe un compromis optim între detalii vizuale și timp de procesare. Fiecare imagine a fost convertită în tensori compatibili cu GPU-ul și normalizată folosind statisticile datasetului ImageNet, pe care fusese antrenat modelul de bază. Această normalizare a asigurat compatibilitatea cu ponderi preantrenate și a redus riscul de dezechilibru în propagarea semnalului prin rețea.

Pentru a crește capacitatea de generalizare a modelului, s-a aplicat augmentarea datelor, prin diverse transformări ale imaginilor, precum rotații, inversări sau variații de luminozitate.. Printre acestea s-au numărat rotații aleatorii, ajustări ale luminozității și contrastului, introducerea unui zgomot gaussian discret și scalări variate ale imaginilor. Scopul acestor augmentări a fost simularea condițiilor imprevizibile din mediul real, unde imaginile pot suferi modificări prin compresie, iluminare neuniformă sau rezoluție redusă.

Modelul Mask R-CNN a fost inițial antrenat pe datasetul COCO, iar ulterior structura de clasificare a fost ajustată pentru a reflecta specificul binar al proiectului („imagini autentice” și

„imagini falsificate”). Această strategie de învățare prin transfer a redus semnificativ timpul de antrenare și resursele necesare, oferind performanțe bune chiar și cu un număr limitat de epoci.

Antrenarea s-a realizat folosind algoritmul Stochastic Gradient Descent (SGD), ales pentru stabilitatea și eficiența sa în cazul arhitecturilor complexe. Parametrii inițiali includeau o rată de învățare redusă, pentru a evita oscilațiile bruște, și un momentum ridicat, pentru a asigura actualizări consistente ale ponderilor. De asemenea, s-a aplicat regularizarea L2 (weight decay) pentru a preveni supraînvățarea. Un mecanism de planificare a ratei de învățare de tip ReduceLROnPlateau a ajustat automat rata de învățare atunci când performanța pe setul de validare nu mai înregistra îmbunătățiri semnificative.

Procesul de antrenare a fost monitorizat atât numeric, cât și vizual. La intervale prestabilite, modelul a generat predicții pe setul de validare, salvate ca imagini cu casete de delimitare și măști de segmentare suprapuse. Aceste vizualizări au oferit o perspectivă calitativă asupra performanței și au permis identificarea cazurilor dificile, sugerând ajustări pentru parametri de antrenare.

Un element esențial a fost integrarea scripturilor COCOeval, care au permis o evaluare standardizată a performanței. Acestea au calculat metrici precum precizia medie pe ansamblu (mAP) și intersecția peste reuniune (IoU) la diferite praguri, oferind o imagine detaliată a capacității modelului de a detecta și segmenta falsurile. Analiza separată pe clasele „autentic” și „falsificat” a evidențiat diferențele de performanță și scenariile problematice, cum ar fi imaginile cu rezoluție scăzută sau falsurile subtile.

1.4. Rezultate experimentale

Rezultatele experimentale obținute în cadrul acestui proiect au confirmat viabilitatea și robustețea abordării propuse. Evaluarea s-a realizat printr-o combinație de metrici cantitative, precum mAP, IoU, acuratețe, precizie, recall și F1-score, dar și prin analize calitative, constând în vizualizarea predicțiilor pe imagini reale și falsificate.

După finalizarea antrenării, modelul a atins o valoare medie a mean Average Precision (mAP) de aproximativ 82% pe setul de validare, ceea ce indică o capacitate solidă de a identifica și segmenta falsurile în condiții variate. De asemenea, valoarea medie Intersection over Union (IoU) a depășit 0,78, demonstrând că măștile generate de model se suprapun cu un grad ridicat de precizie peste regiunile de interes adnotate manual. Pe setul de testare, complet necunoscut modelului, rezultatele au rămas comparabile, ceea ce evidențiază faptul că acesta nu a suferit de overfitting și a reușit să generalizeze eficient. Mai exact, acuratețea globală a fost de aproximativ 84%, precizia a ajuns la 83%, recall-ul la 81%, iar F1-score-ul la 0,82. Aceste valori echilibrate arată că modelul nu este înclinat excesiv nici spre reducerea numărului de falsuri ratate, nici spre minimizarea alarmelor false, ci menține un compromis adecvat între cele două obiective.

Analiza pe fiecare clasă a relevat o diferență notabilă între performanța obținută pentru imaginile „real” și cele „fake”. Pentru clasa „real”, modelul a obținut o precizie de peste 86%, ceea ce indică o capacitate bună de a recunoaște și valida corect imagini autentice. În schimb, pentru clasa „fake”, valorile au fost ușor mai reduse, cu o precizie de aproximativ 80%, fapt ce reflectă dificultatea suplimentară a sarcinii de detectare a falsurilor subtile, mai ales în condițiile imaginilor cu rezoluție scăzută sau cu artefacte vizuale reduse. Această diferență este explicabilă, având în vedere că falsurile moderne generate de tehnici avansate, precum GAN-urile de ultimă generație, pot fi extrem de convingătoare, chiar și pentru un model antrenat pe un volum mare de date.

Pe lângă măsurătorile numerice, analiza vizuală a oferit o confirmare suplimentară a performanței. În numeroase cazuri, modelul a reușit să genereze bounding box-uri corecte și măști de segmentare bine conturate pentru regiunile falsificate ale fețelor. Imaginile cu rezultate corecte au

arătat o suprapunere aproape perfectă între predicțiile automate și adnotările manuale din dataset. Totuși, au existat și situații problematice, în special pentru imaginile cu lumină scăzută sau zgomot vizual ridicat, unde măștile erau incomplete, precum și în cazul falsurilor subtile care afectau doar porțiuni mici ale feței, cum ar fi ochii sau gura, unde modelul tindea să rateze detaliile fine. De asemenea, în scenariile cu multiple fețe, segmentarea era corectă doar pentru subiectul principal, dar mai puțin precisă pentru fețele secundare din imagine.

Aceste observații sugerează direcții de îmbunătățire viitoare, printre care se numără utilizarea unor strategii de fine-tuning pe subseturi dificile și integrarea unor tehnici adiționale, precum post-procesarea cu Soft-NMS pentru reducerea suprapunerii predicțiilor fals pozitive.

Rezultatele experimentale arată că modelul Mask R-CNN, adaptat și optimizat corespunzător, constituie o soluție eficientă pentru detecția și segmentarea deepfake-urilor. Performanța ridicată atât pe setul de validare, cât și pe cel de testare, confirmă potențialul practic al acestei metode în scenarii reale. Totuși, diferența de performanță între clase și dificultățile întâlnite în anumite condiții speciale subliniază faptul că problema deepfake-urilor rămâne una complexă și în continuă evoluție.

2. Work planning

No.	Work activity	Start date	End date
1	State of the art	07.06.2025	09.06.2025
2	Theoretical fundamentals	10.06.2025	20.06.2025
3	Implementation	21.06.2025	10.07.2025
4	Experimental results	11.07.2025	30.07.2025
5	Conclusion	30.07.2025	31.07.2025
6	Appendix	01.08.2025	02.08.2025
7	Summary	03.08.2025	06.08.2025
8	Code	05.03.2025	06.06.2025

3. State of the art

3.1. Project context

Recent developments in deep learning have been applied to create tools that threaten privacy, democratic processes, and national security. One such application is deepfake technology, which enables the generation of highly realistic fake images and videos that are often indistinguishable from genuine content. As a result, there is a critical need for technologies capable of automatically detecting and verifying the authenticity of digital visual media. Addressing this need, researchers have proposed various solutions aimed at identifying manipulated content and ensuring media integrity, while also highlighting the broader implications and challenges posed by the spread of deepfakes [1].

The term “deepfake”, a combination of “deep learning” and “fake,” refers to AI-driven techniques used to manipulate visual content particularly human faces in static images. A common form of image-based deepfake is face-swapping, where the face of a target individual is replaced with that of a source individual, creating the illusion that the target is expressing the same facial features or identity as the source. This approach is frequently referred to as face-swapping deepfake manipulation. Beyond face-swapping, deepfakes in images can also involve other types of AI-generated content. For example, in lip-sync deepfakes, a person’s mouth shape in an image or sequence of images can be altered to match a given audio, giving the impression that the person is speaking the audio content. Another category, often referred to as puppet-master deepfakes, involves projecting the facial expressions, eye movements, and head poses of a source person onto a target’s face, making it appear as though the target is imitating or mimicking the source in real time even in static or frame-by-frame image sequences[2].

While some deepfakes can be created with CGI (computer-generated imagery) or traditional visual effects, deep learning techniques, especially autoencoders and Generative Adversarial Networks (GANs), which are commonly employed in computer vision, are the mainstay of modern deepfake development. After analysing movements and facial expressions, these models create representations of another person with the same attributes. Large image and video datasets are usually needed to train these models efficiently and produce convincing deepfakes. As a result, the main targets are frequently public personalities with a wealth of visual data available online, such as politicians and celebrities [3].

The use of deepfake technology to overlay a famous person's face on another person in a video initially came to the public's notice in 2017, which sparked serious ethical and security issues. Since then, the quickly developing technology has been utilized to produce incredibly lifelike false content featuring public individuals. This presents significant hazards, particularly when world leaders or other powerful people's words or deeds are fabricated through manipulated media, which could lead to the spread of false information and erode public confidence[3].

The rise of advanced deep neural networks, combined with the abundance of data, has made it increasingly difficult for both humans and sophisticated algorithms to distinguish between authentic and forged images or videos. Creating such manipulated content has also become significantly easier, often requiring only a single photo or a short video clip of the target person. With minimal effort, it is now possible to generate highly convincing fabricated footage. In fact, recent technological advancements can even produce deepfakes from a single still image. As a result, deepfakes pose a serious threat not only to public figures but also to everyday individuals. Digitally altered photos and films that contain facial information, especially those created using Deepfake techniques have recently gained a lot of public attention[3] [5].

Deepfakes can be maliciously exploited to incite political or religious conflicts between nations, deceive the public, and influence the outcomes of election campaigns. They also have the potential to disrupt financial markets by spreading fabricated news. In more advanced scenarios, deepfakes could even be used to generate fake satellite imagery, inserting non-existent objects to mislead military analysts, for example, creating the illusion of a bridge over a river where no such structure exists[3], [5].

3.2. Project objectives

Finding the truth in the digital world is now more crucial than ever. This work becomes particularly challenging when dealing with deepfakes, which are frequently produced maliciously and are now available to almost everyone using easily accessible tools. Many other detection methods have been created in response, the majority of which use deep learning. Deep learning technologies are therefore being used to both produce and protect against misleading material [3].

As the realism and complexity of synthetic visual content continue to increase, the challenge of identifying manipulated images particularly those involving human faces generated through artificial intelligence has become more pressing. In recent years, researchers have shifted their focus toward developing robust and precise detection methods aimed specifically at identifying deepfake content. These are not merely minor edits or aesthetic changes to a person's appearance, but fully AI-generated facial forgeries created to deceive viewers by imitating the identity, expressions, and visual features of real individuals. This type of manipulation presents a serious threat in the context of misinformation, identity fraud, and digital trust, especially considering how easily accessible deepfake generation tools have become to the public.

Traditional approaches in digital image forensics, which typically relied on analyzing compression artifacts, noise patterns, or copy-move traces, are no longer sufficient when faced with the high-quality forgeries produced by modern deep learning models such as GANs (Generative Adversarial Networks). Instead, researchers now rely on deep learning-based detection systems that can learn complex patterns and inconsistencies directly from the data. These systems are trained on large datasets of real and fake images and can detect subtle differences introduced during the face synthesis process differences that are often imperceptible to the human eye.

The field has gradually moved from general-purpose image tampering detection to highly specialized techniques focused solely on identifying artificially generated faces. Many of these newer methods are influenced by earlier work in biometric anti-spoofing, which involved distinguishing between real and fake biometric inputs, such as printed photos or video replays in facial recognition systems. Convolutional Neural Networks (CNNs) have become the foundation of modern deepfake detection architectures, allowing for the extraction of detailed facial features and patterns that betray the artificial origin of the images.

The growing global concern about the misuse of deepfake technology has led to the creation of several major research initiatives and collaborative efforts. For example, the MediFor (Media Forensics) program launched by DARPA aimed to develop tools for automatic detection of manipulated media. Similarly, high-profile competitions such as the Deepfake Detection Challenge (DFDC), organized by Facebook, and the Media Forensics Challenge (MFC2018), hosted by NIST, have drawn attention to the importance of this field and encouraged the development of state-of-the-art detection models. These efforts reflect a broader movement toward building reliable, explainable, and trustworthy artificial intelligence tools that can help secure the integrity of digital media in an era increasingly affected by synthetic content [2].

3.3. Problem statement

Despite the quick advancement of deepfake generation and detection methods, current detection models frequently have generalization issues. Even though they typically work well on datasets that are like the ones they were trained on, their accuracy drastically declines when they encounter novel or unfamiliar deepfakes, particularly those that are produced through subtle or unobserved manipulation techniques or those that have undergone platform-level changes like compression and resizing on social media. This disparity significantly reduces their usefulness in practical applications; Therefore, the purpose of this project is to design and implement a practical system that leverages deep learning techniques to detect deepfake images. The focus lies in reproducing and adapting existing detection models to develop a stable and reliable solution capable of handling a variety of data sources, manipulation methods, and media degradation scenarios. Rather than introducing a new theoretical framework, this work aims to consolidate proven methods into a functional detection pipeline, emphasizing performance, generalization, and applicability in real-world conditions [11] .

4. Theoretical Fundamentals

4.1. Foundations of deep learning

Convolutional networks, sometimes referred to as convolutional neural networks or CNNs, are a specific type of neural network designed to process data with a predefined, grid-like architecture. Time-series data are one example, which can be viewed as a 1D grid that collects samples at regular intervals, and image data, which can be viewed as a 2D grid of pixels. In real-world applications, convolutional networks have shown remarkable success. The network uses a mathematical concept known as convolution, as indicated by the name "convolutional neural network." One linear operation is convolution. Convolutional networks are just neural networks with at least one layer that employs convolution rather than general matrix multiplication [7].

Convolution is the term for this process. Typically, an asterisk is used to indicate the convolution operation:

$$s(t) = (s * w)(t) \quad (3.1)$$

Where the parameter t represents time the function $s(t)$ is the original signal, while $w(t)$ is the window function or kernel used for filtering or smoothing.

Convolutions over many axes are frequently used. For instance, we most likely wish to utilize a two-dimensional kernel K if we use a two-dimensional image I as our input:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n) \quad (3.2)$$

Where I is the input (often called the input feature map), K is the kernel (also called the filter), S is the output (called the feature map).

Convolution makes use of three key concepts: parameter sharing, equivariant representations, and sparse interactions, that might enhance a machine learning system.

In contrast to classic neural networks, which have all their outputs connected to every input, convolutional neural networks have sparse interactions, meaning that each output unit only connects to a small portion of the input. To detect features like edges without processing the entire input at once, this is accomplished by utilizing tiny kernels that move over the input (such as an image). This method significantly lowers the computational cost and parameter count. Sparse connections do not employ $m \times n$ parameters and operations; instead, they use $k \times n$, where $k \ll m$. This lowers memory utilization and increases efficiency[7].

The concept of using the same parameters again in various model sections is known as parameter sharing. Each weight in a conventional neural network is used just once to link a particular input to a particular output; it is never utilized again. Convolutional neural networks, on the other hand, apply the same kernel, a tiny filter with common weights, to all of the input. This greatly reduces the number of parameters and enables the network to detect the same feature independent of its position in the input by reusing each weight in the kernel at every spatial point. CNNs are referred to as having tied weights because of this weight reuse, which also enhances their effectiveness and generalizability[7].

Equivariant representations mean that when the input is transformed, the output representation changes in the same way. Formally, for a function f and a transformation g equivariance is defined as:

$$f(g(x)) = g(f(x)) \quad (3.3)$$

The functions f and g are mappings that take an input x and return some output. The equation states that applying g after f gives the same result as applying f after g , meaning the two functions commute for that input x .

4.2. Advanced architectural components

In the context of deepfake forgery segmentation, deep and expressive structures that can handle visual information at numerous spatial resolutions, compress representations, and maintain gradient flow are necessary for effective localization and classification of modified regions. This project's Mask R-CNN system relies heavily on the following elements: Feature Pyramid Networks (FPN), Bottleneck Design, and Residual Learning [1] .

4.2.1. Residual learning framework

Networks have a degradation issue as they get deeper since adding additional layers might result in more training error as well as validation error. This occurs because the optimization environment gets more complicated after overfitting—gradients either disappear or blow up, making it challenging to modify weights in very deep regions. It is inefficient to force a deep stack of nonlinear layers to learn that identity directly if the ideal mapping at a set of layers is near the identity function. Rather, they suggest changing the goal to learn the residual, or the difference between the intended mapping and the identity [12].

It is explicitly allowed for a few stacked layers to suit a residual mapping rather than relying on them to match a specified underlying mapping $H(x)$:

$$F(x) = H(x) - x \quad (3.4)$$

Where $H(x)$ is the desired underlying mapping, $F(x)$ is the residual function to be learned, x is the identity mapping (skip connection).

4.2.2. Analysis of gradient flows

Training very deep networks often suffers from the vanishing gradient problem, where gradients diminish as they are propagated backward, making earlier layers difficult to update. Residual learning addresses this issue by introducing skip connections, which provide a direct path for gradient flow.

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial x_L} \cdot \frac{\partial x_L}{\partial x_1} = \frac{\partial L}{\partial x_L} \cdot \left(1 + \frac{\partial F(x_1)}{\partial x_1}\right) \quad (3.5)$$

Where L is the loss function, x_1 is the input at the first layer, x_L is the output after L layers, and $F(x_1)$ is the residual function. The term “1” ensures that even if the residual mapping’s gradient vanishes, gradients can still propagate through the identity shortcut. This mechanism ensures stable gradient flow in very deep networks and is a key reason why ResNets outperform plain deep CNNs.

4.2.3. Bottleneck architecture theory

Efficiency and expressive power must be balanced in modern networks. Bottleneck blocks, which usually project feature maps into lower-dimensional space via 1×1 convolutions, minimize the number of parameters and processes while maintaining performance [1].

$$Y[i, j, k] = \sigma \left(\sum_c W[c, k] \cdot X[i, j, c] + b[k] \right) \quad (3.6)$$

By striking a balance between prediction and compression, the information bottleneck approach offers a foundation for learning representations. Optimizing the trade-off between maintaining predictive power and condensing the input can be seen as learning effective representations. This is the information bottleneck (IB) goal:

$$I(X; Y) - \beta I(Y; Z) \rightarrow \max \quad (3.7)$$

Where $I(X; Y)$ is the mutual information between input X and representation Y , $I(Y; Z)$ is the mutual information between representation Y and target variable Z , and β is a positive scalar that controls the trade-off between predictive power and compression.

To efficiently extract deep semantic information, the ResNet backbone uses bottleneck blocks. This is especially important when dealing with high-resolution facial data and multi-face pictures, as those included in the OpenForensics dataset. For both training and inference, the network’s memory efficiency is enhanced by reducing dimensionality with 1×1 convolutions [1].

4.2.4. Feature pyramid networks

FPN uses a top-down design with lateral connections to construct an in-network feature pyramid from a single-scale input. A quicker R-CNN with an FPN backbone is used to extract ROI features from different feature pyramid levels according on their scale, but otherwise the rest of the procedure is like plain ResNet. With a ResNet-FPN backbone, mask RCNN feature extraction produces impressive speed and accuracy gains [1].

A top-down pathway with lateral connections is how FPN functions:

$$P_i = \text{Upsample}(P_{i+1}) + C_i \quad (3.8)$$

In this formulation, P_i denotes the feature map at pyramid level i , which is obtained by combining two components. The first component, $\text{Upsample}(P_{i+1})$, corresponds to the higher-level feature map P_{i+1} , which is spatially coarser but semantically richer, upsampled to match the resolution of the current level. The second component C_i , represents the lateral feature map extracted from the backbone network at the same level i , usually after applying a 1×1 convolution to adjust its channel dimensions. By summing these two contributions, each pyramid level P_i integrates both high-level semantic information and fine-grained spatial detail, leading to a multi-scale feature hierarchy that is well-suited for object detection.

4.3. Object detection

4.3.1. Comparing one-stage and two-stage detection methods

A detection technique that applies classifiers and regressors on sliding windows across convolutional feature maps was introduced in the OverFeat study [13]. This represents a class-specific, one-stage detection method. In contrast, two-stage detection pipelines first generate class-agnostic region proposals, followed by class-specific object recognition. To extract region-level information, OverFeat employs a sliding window approach over a scale pyramid with a fixed aspect ratio. Using these features, it simultaneously predicts object localization and classification.

By comparison, the Region Proposal Network (RPN) uses information from 3×3 sliding windows to predict offsets relative to anchor boxes of different scales and aspect ratios, thereby generating candidate regions. In the two-stage RPN + Fast R-CNN framework, this constitutes only the first step; the proposals are refined in the second stage using a more accurate detector. Both OverFeat and RPN rely on sliding windows, but the latter integrates anchor-based proposals to improve detection flexibility.

For a fair comparison between one-stage and two-stage approaches, previous studies have configured Fast R-CNN in a one-stage mode to simulate OverFeat. In this setting, Fast R-CNN predicts class scores and refines bounding box coordinates by densely sampling sliding windows with three aspect ratios (1:1, 1:2, 2:1) and three scales (128, 256, 512) [14] [13].

4.3.2. Region proposal networks (RPN) theory

A Region Proposal Network (RPN) generates a series of rectangular object suggestions, each with an objectness score, from an input image of any size. In this section, we describe a fully convolutional network that we use to mimic this process. Since sharing computation with a Fast R-CNN object detection network is our ultimate objective, we presume that both nets share the same convolutional layer set [14].

A lower-dimensional feature vector is assigned to each sliding window, with 256 dimensions for the ZF model (Zeiler and Fergus network) and 512 dimensions for the VGG model (Visual Geometry Group network). This vector is processed by two parallel fully connected layers: a box-classification layer (cls) and a box-regression layer. Given that the effective receptive field on the input image is large (171 pixels for ZF and 228 pixels for VGG), the architecture typically uses $n=3$. In practice, this design is implemented with two parallel 1×1 convolutional layers following an $n \times n$ convolutional layer. The output of the $n \times n$ convolution is then passed through ReLU activation functions. [14].

A sigmoid function is used to classify each anchor as either foreground (object) or background (non-object), and cross-entropy is used to define the binary classification loss:

$$L_{\text{cls}} = -\frac{1}{N_{\text{cls}}} \sum_i [p_i^* \log(p_i) + (1 - p_i^*) \log(1 - p_i)] \quad (3.9)$$

In this formulation, p_i represents the predicted probability that anchor i contains an object, while p_i^* denotes the corresponding ground-truth label, which takes the value 1 for positive anchors and 0 for negative anchors. The first term, $p_i^* \log(p_i)$, enforces high confidence for correctly classified positive anchors, the second term, $(1 - p_i^*) \log(1 - p_i)$, enforces high confidence for correctly classified negative anchors. The normalization factor N_{cls} corresponds to the number of

anchors considered in the mini batch, ensuring that the magnitude of the loss remains consistent regardless of batch size. In this way, the cross-entropy formulation penalizes incorrect classifications and drives the model to assign higher probabilities to the correct anchor labels.

4.3.3. Non-Maximum suppression (NMS) algorithm

To minimize redundant bounding boxes in object detection, Non-Maximum Suppression (NMS) is an essential post-processing step. NMS only chooses the boxes with the greatest scores and discards the others based on a threshold over Intersection over Union (IoU), since several boxes frequently overlap the same object [14].

The spatial overlap between two bounding boxes, A and B, is measured by IoU, which is defined as follows:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (3.10)$$

Greedy algorithms divide a problem into numerous parts and then determine the best version of each part separately. Regretfully, putting the individually ideal elements together does not always result in an ideal whole answer. The quality of a greedy answer is frequently adequate, if not ideal, and greedy algorithms can be computationally far less expensive than algorithms that search for the best joint solution. A fine-tuning step, where a joint optimization algorithm looks for the best solution to the entire problem, may also come after greedy methods. The joint optimization algorithm can be significantly accelerated and the quality of the solution it discovers can be enhanced by starting it with a greedy solution. Deep learning relies heavily on pretraining algorithms, particularly greedy pretraining methods. This section focuses on pretraining methods that decompose supervised learning issues into more manageable supervised learning tasks issues. We call this method greedy supervised pretraining [7] [15].

4.4. Mask R-CNN

Mask R-CNN is straightforward: for every candidate item, Faster R-CNN produces two outputs: a class label and a bounding-box offset. It is then add a third branch that produces the object mask. Thus, Mask R-CNN is a logical and intuitive concept. However, the extra mask output is different from the class and box outputs, necessitating the extraction of an object's considerably finer spatial arrangement. The essential components of Mask R-CNN are then presented, including pixel-to-pixel alignment, the primary component that Fast/Faster R-CNN lacks [10].

Mask R-CNN uses the same two-step process, starting with the same RPN stage. In the second phase, concurrent with class and box prediction mask, Mask R-CNN additionally produces a binary mask for every ROI. In contrast, most current systems rely on mask predictions for categorization. Our method adheres to the principles of Fast R-CNN, which uses bounding-box classification and regression in parallel and greatly simplifies the original R-CNN's multi-stage pipeline [10].

It is formally defined a multi-task loss on each sampled ROI during training as $L = L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}$. The bounding-box loss L_{box} and classification loss L_{cls} are the same as those specified in. The Km2 of the mask branch is dimensional output that encodes K binary masks with a resolution of $m \times m$, one for each of the K classes, for every ROI. We apply a per-pixel sigmoid to this, and the average binary cross-entropy loss is defined as L_{mask} . L_{mask} is only specified on the k-th mask for a ROI linked to ground-truth class k; other mask outputs do not affect the loss [10].

4.4.1. R-CNN

Bounding-box object detection using the Region-based CNN (R-CNN) approach involves evaluating convolutional networks independently on each ROI while focusing on a manageable amount of potential object regions. To handle RoIson feature maps with RoIPool, R-CNN was expanded, which resulted in faster processing times and increased accuracy. This stream was upgraded by Faster R-CNN, which used a Region Proposal Network (RPN) to learn the attention mechanism. Faster R-CNN is currently the top framework in a number of benchmarks and is adaptable and resilient to numerous follow-up modifications [10].

Many methods for instance segmentation are based on segment proposals, which are motivated by the efficacy of RCNN. Previous approaches used bottom-up segments. Segment candidates are proposed by DeepMask and subsequent efforts, and Fast R-CNN classifies them. In these techniques, segmentation comes before recognition, which is less precise and takes longer [10].

The spatial arrangement of an input object is encoded by a mask. Therefore, in contrast to box offsets or class labels, which are invariably condensed into brief output vectors by w it fully connected (fc) layers, the pixel-to-pixel connectivity offered by convolutions makes it easy to recover the spatial structure of masks. An FCN is employed to predict an $m \times m$ mask from each ROI [10]. This approach prevents the layers in the mask branch from collapsing into a vector representation that lacks spatial dimensions, while preserving the explicit $m \times m$ spatial structure of the object. Fully convolutional representations use fewer parameters and have been demonstrated experimentally to achieve higher accuracy compared to earlier methods that relied on fully connected layers for mask prediction[10].

Because of this pixel-to-pixel behaviour, our ROI features—which are tiny feature maps in and of themselves, must be precisely aligned to maintain the explicit per-pixel spatial relationship. The need to maintain precise per-pixel spatial alignment in the ROI features led to the development of the RoI Align layer, which is crucial for accurate mask prediction [10].

A common procedure for obtaining a tiny feature map (such as 7×7) from every ROI is RoIPool. Prior to aggregating the feature values covered by each bin, RoIPool quantizes a floating-number ROI to the feature map's discrete granularity. This quantized ROI is then separated into spatial bins, which are quantized as well. Quantization is carried out, for example, by computing $\lfloor x/16 \rfloor$ on a continuous coordinate x , where 16 is a feature map stride and $\lfloor \cdot \rfloor$ is rounding; quantization is also carried out when splitting into bins, such as 7×7 . The ROI and the extracted features become misaligned because of these quantizations. This has a significant detrimental influence on pixel-accurate mask prediction, but it may not influence categorization, which is resilient to slight translations [10].

4.5. Loss functions and optimization

4.5.1. Multi-Task learning

Deepfake segmentation is not a single problem; rather, it is a collection of related tasks that must be solved concurrently. Classifying modified versus legitimate content, identifying and localizing objects using bounding boxes, and creating pixel-level segmentation masks to indicate manipulated face regions are typical examples of these tasks. In addition to being ineffective, tackling these activities separately could result in inconsistent or less-than-ideal outcomes because

there would be no common contextual understanding. Multi-Task Learning (MTL) becomes crucial in this situation [16].

A single model is trained to do several related tasks at once using the MTL paradigm of machine learning. As with classification, object identification, and segmentation in deepfake analysis, the main notion is that these tasks might benefit from a shared representation, particularly when they are complementary in nature. MTL enhances the model's capacity for generalization, lowers the possibility of overfitting, and even makes up for sparse or noisy labels—which are typical in real-world deepfake datasets by utilizing similarities and differences among these tasks [16].

According to theory, multi-task learning is created by integrating the loss functions related to every task into a single loss function. By allowing the model to learn a set of parameters that concurrently minimize the error for all tasks involved, this composite loss function directs the learning process [16]. The mathematical expression for the total loss function is:

$$L_{\text{total}}(\theta) = \lambda_{\text{cls}} \cdot L_{\text{cls}} + \lambda_{\text{bbox}} \cdot L_{\text{bbox}} + \lambda_{\text{mask}} \cdot L_{\text{mask}} \quad (3.11)$$

Where θ represents the shared model parameters, L_{cls} is the loss associated with image-level classification, L_{bbox} corresponds to the regression loss for bounding box localization, L_{mask} is the per-pixel segmentation loss, and $\lambda_{\text{cls}}, \lambda_{\text{bbox}}, \lambda_{\text{mask}}$ are scalar weighting coefficients that balance the contribution of each task to the overall learning objective as shown in equation (3.11).

4.5.2. Classification loss

Classification is essential to the Mask R-CNN framework since it establishes the category of every object or area that is detected. This frequently refers to determining whether an identified facial region is "real" or "fake" in the context of deepfake segmentation. The learning goal is to reduce the difference between the anticipated distribution and the ground truth, which is produced by the network's classification head as a probability distribution across the potential classes [10].

Cross-entropy loss, a common loss function in classification problems, is used to quantify this dissimilarity. Predictions that differ from the actual label distribution are penalized. The categorical cross-entropy loss for a generic multi-class situation is provided by [10]:

$$L_{\text{cls}} = - \sum_{i=1}^C y_i \log(p_i) \quad (3.12)$$

Where C is the number of classes, y_i is the one-hot encoded true label for class i , and p_i is the predicted probability for class i as defined in equation (3.12).

4.5.3. Mask loss

A key element of instance segmentation in deepfake detection is mask prediction, which aims to produce a binary mask that accentuates the altered areas at the pixel level for every identified face. Every pixel in the mask represents either modified (1) or authentic (0), and this is formulated as a pixel-wise binary classification issue [10].

Binary cross-entropy is used to calculate the mask loss for each pixel in the expected mask separately:

$$L_{\text{mask}} = \frac{1}{m^2} \sum_{i=1}^m \sum_{j=1}^m [y_{ij} \log(\hat{y}_{ij}) + (1 - y_{ij}) \log(1 - \hat{y}_{ij})] \quad (3.13)$$

Where m^2 is the resolution of the predicted mask, $y_i \in \{0,1\}$ is the ground-truth value at pixel (i,j) , and $\widehat{y}_{ij}$ is the predicted sigmoid output at pixel (i,j) .

4.6. Optimization algorithms

To minimize loss functions across intricate, high-dimensional spaces, modern deep learning models mostly rely on effective and theoretically supported optimization procedures. The fundamental and sophisticated optimization methods that make up the training pipeline of deepfake segmentation systems, like Mask R-CNN structures, are covered in detail in this section. In addition to convergence efficiency, the methods discussed here also address generalization robustness and adaptability across deep learning's intrinsic non-convex objectives [7] [10].

By updating the parameters in the opposite direction as the function's gradient with respect to the parameters, the iterative first-order optimization process known as "Gradient Descent" minimizes an objective function. It works in both convex and non-convex environments and is predicated on the goal function's differentiability [10].

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) \quad (3.14)$$

Where $\theta_t \in \mathbb{R}_n$ is the model parameter vector at iteration t , $\eta > 0$ is the learning rate (step size), and $\nabla L(\theta_t)$ is the gradient of the loss function at θ_t .

The adaptive gradient-based optimization technique Adam (Adaptive Moment Estimation) is frequently used in deep learning due to its low memory needs and high computing efficiency. The two previous approaches, RMSprop (which uses exponential moving averages to reduce noisy updates) and AdaGrad (which uses per-parameter adaptive learning rates to manage sparse gradients), are combined in this method. Adam uses bias correction to stabilize updates in the early iterations and keeps a running average of the gradients' first moment (mean) and second moment (uncentered variance) [17].

A dynamic learning rate scheduling method called cosine annealing lowers the learning rate after a cosine function, allowing for progressive convergence while preserving the ability to avoid abrupt or less-than-ideal minimum. It works especially well for fine-tuning in the latter phases of training [17].

With a probability of p , dropout is a stochastic regularization technique that randomly "drops" neurons (sets their output to zero) during training. This effectively trains an ensemble of sub-networks and reduces the possibility of neuronal co-adaptation. All units are employed during inference, and training-time dropout is taken into account by scaling their outputs [17].

A thorough and exacting framework for evaluating the performance of models entrusted with object detection, instance segmentation, key point detection, and panoptic segmentation is represented by the COCO (Common Objects in Context) assessment metrics [8].

4.6.1. COCO evaluation

These criteria, which were first introduced as part of the Microsoft COCO dataset challenge, are now widely used in both academia and industry to assess contemporary computer vision systems. In contrast to conventional classification metrics that only consider prediction correctness, the COCO assessment system takes instance-level precision, spatial accuracy, and localization

confidence into account. By integrating elements of pixel-level mask segmentation, bounding box localization, and object categorization, it offers a comprehensive viewpoint on model quality [8].

The purpose of COCO metrics in object detection and segmentation is to evaluate not only the correctness of object identification but also the accuracy of spatial extents and object localization in the image. The mean Average accuracy (mAP), the primary metric utilized in COCO, is calculated by averaging accuracy values across several Intersection over Union (IoU) criteria in steps of 0.05, ranging from 0.50 to 0.95. With this architecture, models are rewarded for their capacity to give accurate and high-fidelity localization rather than just for rough localization, as is the case with a set IoU threshold of 0.5 [8].

The metric suite provides information about how effectively a model generalizes across diverse real-world circumstances by assessing detection performance across a range of item sizes (small, medium, and big) and counts (e.g., AP for 1–10, 10–100, and 100+ detections). In situations when accurate localization and segmentation are difficult, such as in scenes or photos with clutter, this is especially crucial. Furthermore, by differentiating between Average Precision (AP) and Average Recall (AR), COCO metrics provide for a more comprehensive assessment of a model's performance than just one summary number. While AR shows the maximum recall feasible given a constant number of detections per image, AP assesses how well a model can balance precision and recall at various confidence thresholds [8].

The capacity of COCO assessment measures to standardize performance comparisons across various model architectures and research initiatives is what makes them significant. Many cutting-edge models, like Mask R-CNN, RetinaNet, YOLOv5, and DETR, which are often evaluated against COCO criteria, have been impacted by them in their design and benchmarking. Its broad acceptance is further supported by the official COCO API, which offers a transparent and repeatable process for evaluations [8].

4.6.2. Mathematical formulation of COCO evaluation metrics

The Intersection over Union (IoU), which quantifies the spatial overlap between the associated ground truth and the projected region (bounding box or mask), is the central component of the COCO metric suite. The area of junction divided by the area of union is known as the area of union (IoU)[18]:

$$\text{IoU} = \frac{|\text{Prediction} \cap \text{Ground Truth}|}{|\text{Prediction} \cup \text{Ground Truth}|} \quad (3.15)$$

The assessment is based on the traditional definitions of recall and precision that have been modified for object detection. The ratio of accurate detections (true positives) to all model predictions (true + false positives) is known as precision. The percentage of true positives found out of all ground truth occurrences is known as recall [18].

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.16)$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where TP is true positives, FP is false positives, and FN is false negatives.

COCO determines Average Precision (AP) as the area under the precision-recall curve at a fixed IoU threshold t in order to assess performance across the entire range of model confidence levels [18] :

$$AP_t = \int_0^1 p(r) dr \quad (3.17)$$

In contrast to other benchmarks (like PASCAL VOC), which employ a single IoU threshold (like 0.5), COCO rewards exact localization by averaging AP across ten thresholds, ranging from 0.50 to 0.95 in 0.05 increments. A common name for the resulting statistic is AP [.50:.95] [18]:

$$mAP = \frac{1}{10} \sum_{t=0.50}^{0.95} AP_t \quad (3.18)$$

with $t \in \{0.50, 0.55, \dots, 0.95\}$

COCO also offers Average Recall (AR) scores, which quantify the maximum recall for a predetermined number of top-scoring detections each image, in addition to AP. This metric is calculated and averaged using the same set of IoU thresholds as AP [18]:

$$AR_{\max=k} = \frac{1}{10} \sum_{t=0.50}^{0.95} Recall_{t,k} \quad (3.19)$$

for $k \in \{1, 10, 100\}$

5. Implementation

The deepfake segmentation system is implemented in this chapter utilizing a modified Mask R-CNN with a ResNet-50-FPN backbone, pretrained on COCO and optimized for background, fake, and real classes. The OpenForensics dataset is used to train and assess the model. It offers pixel-level annotations for both real and altered facial regions in a variety of naturalistic settings. Data loading in the COCO style, mask preprocessing, performance assessment based on IoU and class accuracy, and training and visualization adjustments made for GPU resource constraints are all included in the implementation.

5.1. Data usage

The integration of the COCO-format annotation file, which contains comprehensive metadata like image file paths, bounding boxes, segmentation masks, and their corresponding category labels, is the first step in the dataset preparation process. The pycocotools library, which provides a structured API for accessing and exploring COCO data, is used to parse these annotations. Multiple object instances, each providing geographical and categorical annotation data, are associated with each image. This organized format is ideal for applications involving intricate image segmentation because it facilitates efficient filtering, retrieval, and processing.

After loading the annotations, a verification step is carried out to confirm the validity of the dataset. This involves checking whether the image files referenced in the annotation file are physically present in the dataset directory. Any references to missing or corrupted files are removed at this stage, which prevents interruptions or crashes during model training and ensures that the dataset used is clean and reliable. Once the dataset is validated, it is split into two subsets: one for training and the other for validation. Importantly, the split is done at the image level to preserve the relationships between each image and its complete set of annotations, including all associated bounding boxes and masks. This is essential in segmentation tasks, where spatial consistency is critical.

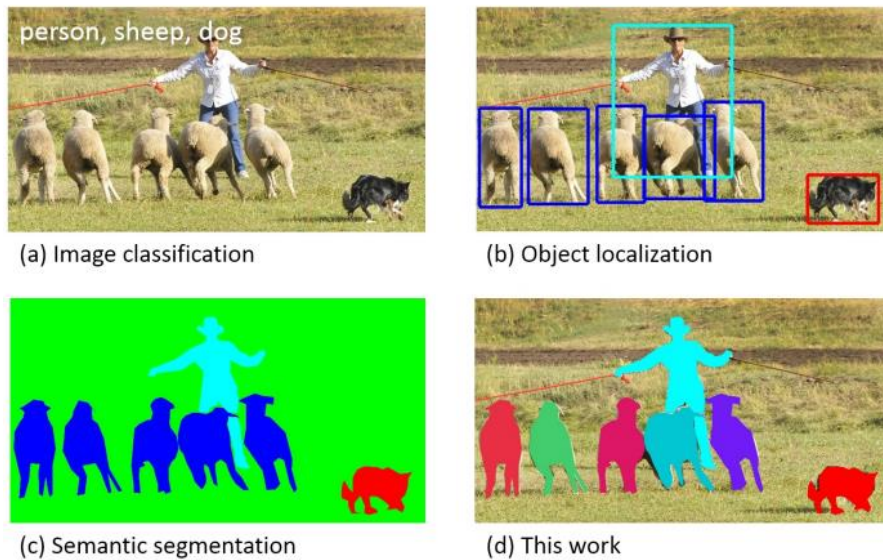


Figure 5.1: Computer vision tasks applied to COCO dataset

Figure 5.1 demonstrates the progression of four key computer vision tasks using a pastoral scene from the COCO dataset, showing a person herding sheep with a dog. This example highlights how different computational approaches capture varying levels of detail and spatial understanding from the same image [8].

Image classification, shown in panel (a), represents the simplest form of visual understanding. It identifies the objects present in the image, such as a person, sheep, and dog, without providing any information about their locations [8].

Object localization, presented in panel (b), adds spatial awareness through bounding boxes. Rectangles are drawn around each detected object: the person in cyan, the sheep in blue, and the dog in red. This provides both recognition and approximate location information, similar to COCO-style object detection annotations [8].

Semantic segmentation, illustrated in panel (c), assigns each pixel in the image to a specific class. The person is shown in cyan, the dog in red, the sheep in blue, and the background in green. In this approach, all objects of the same class are treated as one, so individual sheep cannot be distinguished from each other [8].

Instance segmentation, displayed in panel (d) as "This work," represents the most advanced level of analysis. It combines the benefits of object detection and semantic segmentation by generating pixel-accurate masks for each individual object. Each sheep receives a unique color, such as red, green, magenta, cyan, and purple. This allows the system to separate individual sheep while maintaining precise pixel-level accuracy, demonstrating COCO-style instance annotations that identify, locate, and differentiate every object in a complex scene [8].

A label mapping technique is used to expedite the categorization of segmentation masks once the dataset has been separated. In this configuration, all other valid COCO category IDs are regarded as authentic, whereas COCO category ID is utilized to identify altered information. Any area that isn't annotated is automatically categorized as background and given the label 0. As a result, the learning task becomes easier to handle while maintaining semantic separation between classes thanks to a simplified structure background, fake, and real.

The next step involves establishing the label mapping system for the segmentation task. Category ID from the COCO annotations is designated as fake content, while all other valid category IDs are classified as real content. Background areas, which are not covered by any annotation, are implicitly assigned label 0. This three-class mapping (background, fake, real) simplifies the training process while maintaining clear semantic distinctions.

To improve training reliability, the dataset class includes built-in error handling. If any image or its annotations are unreadable or corrupted, the class returns a Nonvalue. These are automatically filtered out during batch preparation by a custom `collate_fn` function in the PyTorch DataLoader, thus ensuring uninterrupted data flow even in the presence of problematic samples [6].

For efficiency, the data loading process avoids preloading the entire dataset into memory. Instead, images are processed dynamically as needed, reducing VRAM usage. The DataLoader is optimized with parameters such as `pin_memory` and multi-threaded loading to speed up data transfer to the GPU, which is especially beneficial when operating with mid-range hardware like the NVIDIA GTX 1650 Ti used in this research.

Although the class does not apply data augmentation directly, it is fully compatible with external transformation pipelines using PyTorch's transform module. This flexible design allows developers to experiment with various augmentation techniques without altering the core dataset logic [6].

In the final stage, the three-class label system is consistently applied to every processed sample. This structured mapping ensures semantic consistency throughout the training pipeline and supports both detection and segmentation objectives. The result is a resilient and adaptable dataset

loading architecture, offering solid performance, memory efficiency, and robustness even on constrained computing environments.

The full dataset loading pipeline, including the preprocessing and custom collate function, is presented in Appendix 1.

5.2. Model

To establish a reference point for deepfake segmentation, we initially designed a baseline convolutional neural network (CNN). The architecture followed a conventional layered scheme, where convolutional blocks each consisting of convolution, batch normalization, and ReLU activation were interleaved with pooling layers to progressively capture higher-level features. Deeper layers employed a greater number of filters to extract richer semantic representations, while dropout and L2 regularization were incorporated to mitigate overfitting. The network concluded with a global average pooling layer and fully connected layers, producing softmax-based predictions across three classes [12].

Although this CNN provided a conceptual benchmark, in practice our experiments concentrated exclusively on the Mask R-CNN framework, which offered superior segmentation capabilities and more precise localization of manipulated facial regions. Computational resource constraints also influenced this decision, as the baseline CNN was not deployed beyond preliminary design.

5.2.1. Mask R-CNN

The primary detection system utilizes Mask R-CNN integrated with ResNet-50 backbone networks and pyramid feature architectures, specifically configured for multi-category deepfake segmentation applications. This framework was selected for its capacity to concurrently execute object localization, classification, and instance-level segmentation, providing comprehensive tools for identifying and precisely localizing manipulated facial areas within complex visual scenes [12].

The Mask R-CNN framework incorporates three primary modules: the region proposal generator, detection heads for classification and localization, and mask generation components. The proposal network identifies potential object regions through systematic image analysis. Detection modules then categorize these regions (authentic versus synthetic) while refining spatial boundaries. The mask generation component produces pixel-level segmentation maps for each identified object, enabling precise localization of manipulated areas. Joint training of all components uses composite loss functions incorporating classification, localization, and segmentation objectives [10].

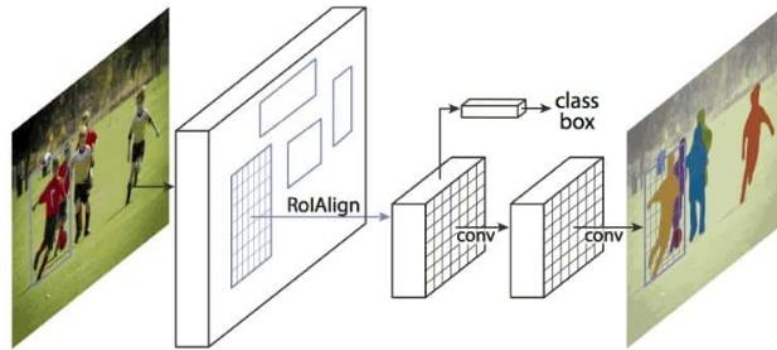


Figure 5.2 : The Mask R-CNN framework for instance segmentation

Mask R-CNN is a cutting-edge framework that extends the Faster R-CNN model by incorporating an additional branch dedicated to predicting segmentation masks for each Region of Interest (RoI), alongside the traditional branches for classification and bounding box regression. This combination of object detection and instance segmentation makes it particularly effective for deepfake detection, allowing precise identification and localization of manipulated facial regions within complex visual scenes. As one can observe in Figure 5.1, the architecture uses RoIAlign, which addresses the quantization limitations inherent in conventional RoI pooling. RoIAlign extracts fixed-size feature maps from regions of varying dimensions while preserving spatial accuracy, which is crucial for high-fidelity mask prediction. The method employs bilinear interpolation to estimate feature values at regularly sampled points, followed by aggregation through max or average pooling, thereby maintaining exact pixel-level alignment between input images and network outputs [10].

Mask R-CNN processes the RoIAlign output through three specialized branches. The classification branch predicts class probabilities for detected objects, distinguishing between real faces, synthetic faces, and background, using fully connected layers with softmax activation. The bounding box regression branch refines object localization by regressing four coordinates per box and employs smooth L1 loss during training. The mask branch generates pixel-level masks for each detected instance via a compact fully convolutional network, producing binary masks for each class and accurately delineating manipulated regions [10] [14].

The model's multi-task loss function combines three components, with total loss defined as the sum of classification, bounding box, and mask losses. Classification loss is calculated using cross-entropy, bounding box regression employs smooth L1 loss, and mask prediction relies on binary cross-entropy. This joint optimization enables the network to simultaneously learn detection and segmentation tasks effectively. Training is performed on standardized 256×256 input images for batch consistency, with annotations converted from polygon-based representations to binary masks using nearest-neighbor resampling. The implementation leverages COCO pre-trained weights for transfer learning, while the final classification layer is adapted specifically to deepfake detection [10].

5.2.2. ResNet-50

ResNet-50 forms the architectural foundation, implementing residual connections to facilitate training of deep network structures. Skip connections mitigate gradient degradation issues while enabling the extraction of sophisticated visual representations. The pyramid feature network augments this foundation by creating multi-resolution feature hierarchies, combining fine-grained spatial details from shallow layers with abstract semantic information from deeper network levels through top-down processing and cross-layer connections. This generates a structured feature pyramid enabling detection and segmentation of objects across various scales and complexities [9].

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Figure 5.3 : ResNet Architectures

ResNet-50 acts as the feature extraction backbone within Mask R-CNN, providing hierarchical feature representations essential for accurate deepfake detection. This architecture comprises 50 layers, including convolutional layers, batch normalization, and fully connected components, as summarized in Figure 5.2, which presents the detailed layer configuration for different ResNet variants [9].

The primary innovation in ResNet lies in residual connections, which mitigate the vanishing gradient problem in deep networks. Rather than learning a direct mapping $H(x)$, residual blocks learn the residual function $F(x) = H(x) - x$, simplifying optimization and enabling very deep networks to be trained effectively. Residual connections are implemented using convolution blocks and identity blocks. Convolution blocks handle dimension mismatches between input and output via 1×1 convolutions with appropriate stride, while identity blocks add the input directly to the output when dimensions match.

The layer configuration begins with conv1, producing a 112×112 output through 7×7 convolutions with stride 2, performing initial feature extraction and spatial downsampling, followed by batch normalization and ReLU activation. The conv2_x stage preserves a 56×56 spatial resolution while building feature representations through 3×3 convolutions with 64 channels repeated twice, forming the first residual block stage.

The conv3_x stage reduces spatial resolution to 28×28 while increasing channel depth to 128. This stage is crucial for mid-level feature extraction, capturing more complex patterns than edges or textures. The conv4_x stage further reduces the spatial size to 14×14 and expands channels to 256, focusing on high-level semantic features essential for object recognition. Finally, the conv5_x stage operates at 7×7 resolution with 512 channels, repeated three times, providing the highest level of feature abstraction for classification tasks. Each stage progressively halves spatial dimensions while doubling channel depth, forming a hierarchical feature map that encodes both fine-grained details and high-level semantics.

ResNet-50 integrates with Feature Pyramid Networks (FPN) through a bottom-up pathway that follows the standard ResNet forward pass, extracting features at multiple scales from conv2_x to conv5_x. The top-down pathway fuses high-level semantic features with low-level, high-resolution features through upsampling and lateral connections, creating pyramid levels P2, P3, P4, and P5 for multi-scale detection.

From a computational perspective, ResNet-50 strikes a balance between accuracy and efficiency. It requires 3.8×10^9 FLOPS, slightly higher than ResNet-34's 3.6×10^9 FLOPS, while offering significantly improved accuracy. This balance makes it suitable for real-time deepfake detection where both speed and precision are critical. Transfer learning leverages pre-trained COCO weights, keeping lower convolutional layers frozen to preserve generic feature representations such as edges and textures, while higher detection and segmentation modules are fine-tuned for manipulation detection. The classification head is modified for three-category segmentation, distinguishing authentic, synthetic, and background regions.

5.2.3. Integrated system architecture

The combination of Mask R-CNN with ResNet-50 creates a comprehensive system for deepfake detection where ResNet-50 provides robust feature extraction across multiple scales while the Feature Pyramid Network enables detection of manipulations at various sizes. The RoIAlign component ensures precise spatial localization, and the three-branch design enables simultaneous classification, localization, and segmentation of manipulated regions.

The training protocol implements batch processing with 256×256 standardized inputs for consistency, multi-task learning through joint optimization of classification, localization, and segmentation objectives, and comprehensive data augmentation where polygon annotations are converted to precise binary masks. Validation strategy employs accuracy and loss metrics across validation datasets, assessing both classification and segmentation performance to ensure robust deepfake detection capabilities.

Performance evaluation demonstrates that the integrated architecture supports future enhancements including attention mechanisms that can be integrated into backbone or detection heads, transformer components for potential replacement or augmentation of CNN components, advanced feature fusion through enhanced pyramid network designs, and real-time optimization techniques such as architecture pruning and quantization. The modular design ensures that improvements can be incorporated without fundamental architectural changes, maintaining the system's adaptability to evolving deepfake generation techniques.

5.3. Training

The training implementation employs a sophisticated custom training loop specifically designed for instance segmentation-based deepfake detection. The architecture implements a three-class classification system with explicit class mapping: background (0), fake faces (1), and real faces (2). This multi-class approach enables the model to distinguish between non-facial regions and the two critical categories in deepfake detection scenarios.

The training process begins with comprehensive dataset preparation and validation. The implementation includes robust error handling and memory management specifically optimized for GPU-constrained environments, particularly targeting systems with limited VRAM such as the GTX 1650 Ti with 4GB memory. The full code is presented at Appendix 2, which demonstrates the complete dataset initialization and path verification process. Prior to training initiation, the system performs thorough validation of all required file paths and generates detailed statistics about the training and validation datasets to ensure data integrity.

The training loop incorporates several memory optimization techniques critical for resource-constrained environments. Mixed precision training is implemented using `torch.cuda.amp.autocast()` to reduce memory consumption while maintaining training stability. The full code is presented at

Appendix 3. This approach allows the model to utilize 16-bit floating-point operations during forward passes while maintaining 32-bit precision for gradient computations, effectively reducing memory requirements by approximately 50% without sacrificing training quality.

The implementation continuously monitors GPU memory allocation throughout the training process and provides proactive warnings when memory usage exceeds safe thresholds, specifically targeting 3.5GB for 4GB GPU configurations. Dynamic memory management includes strategic cache clearing at regular intervals and automatic batch size adjustment recommendations when memory constraints are encountered. The gradient scaling mechanism ensures numerical stability during mixed precision training while the automatic cache clearing prevents memory fragmentation that could lead to out-of-memory errors during extended training sessions.

The training process utilizes the Mask R-CNN loss function, which combines multiple loss components including classification loss for distinguishing between fake and real faces, bounding box regression loss for accurate face localization, and mask segmentation loss for precise pixel-level segmentation. The optimizer configuration employs AdamW with weight decay regularization and learning rate scheduling for improved convergence characteristics. The full code is presented at Appendix 4.

The optimization strategy utilizes AdamW optimizer with a carefully tuned learning rate of 1×10^{-4} , and weight decay of 5×10^{-4} , to prevent overfitting while maintaining stable convergence. The ReduceLROnPlateau scheduler automatically adjusts the learning rate when validation loss plateaus, reducing the learning rate by a factor of 0.5 after three epochs without improvement. Gradient clipping with a maximum norm of 1.0 prevents gradient explosion, which is particularly important when training on diverse datasets with varying image qualities and deepfake generation techniques. The early stopping mechanism monitors validation performance with configurable patience settings to prevent overfitting and reduce unnecessary computational overhead.

The training implementation incorporates a multi-faceted evaluation system that monitors model performance across multiple dimensions. The evaluation process calculates intersection over union (IoU) scores for mask quality assessment and class-specific accuracy metrics for both fake and real face classifications. The full code is presented at Appendix 5. The IoU calculation compares predicted masks against ground truth annotations using a configurable threshold of 0.5, providing quantitative assessment of segmentation quality.

The training system implements comprehensive checkpoint management to ensure training continuity and model preservation. Multiple model states are maintained throughout the training process, including the best validation loss model, best IoU model, and regular training checkpoints. The full code is presented at Appendix E. This multi-model approach ensures that optimal performance can be recovered from different perspectives, whether prioritizing overall loss minimization or segmentation quality maximization.

The checkpoint system preserves complete training states including model parameters, optimizer states, learning rate schedules, and comprehensive metrics history. This approach enables seamless training resumption from any point, allowing for interrupted training sessions to continue without loss of progress. The automatic saving mechanism triggers at configurable intervals and upon achieving new performance milestones. Metrics history preservation includes training and validation losses, IoU scores, class-specific accuracies, and temporal performance trends, providing comprehensive documentation of the training progression for post-training analysis and research reporting.

The incorporation of the background class (class 0) represents a critical design decision for robust deepfake detection capabilities. In instance segmentation tasks, the background class enables the model to distinguish between regions of interest, specifically faces containing either deepfake or

real content, and irrelevant image areas such as scenery, clothing, hair, or other objects that frequently appear in portrait photographs and video frames.

The three-way classification approach creates more robust decision boundaries compared to binary classification systems by reducing the likelihood of misclassifying background pixels as either fake or real faces. This architectural choice enhances feature representations during training by forcing the model to learn discriminative features that clearly separate facial regions from background content. The explicit background modeling minimizes noisy predictions that could occur when background content is misinterpreted as facial features, thereby improving overall detection accuracy in real-world deployment scenarios where images contain diverse and unpredictable background content.

The training implementation includes comprehensive visualization capabilities for monitoring training progress and analyzing model performance throughout the learning process. The visualization system generates multi-panel plots showing training and validation metrics over time, including loss curves that track the convergence behavior, IoU progression that demonstrates segmentation quality improvements, and class-specific accuracy trends that reveal model learning patterns for fake and real face detection. The full code is presented at Appendix 6.

The visualization system provides real-time training progress monitoring with epoch-by-epoch metrics display, enabling researchers to identify potential training issues as they occur. Comparative analysis between training and validation performance helps detect overfitting or underfitting scenarios early in the training process. The system generates class-specific accuracy visualizations that highlight model performance on fake versus real face detection, revealing any class imbalance issues or bias toward categories.

The comprehensive plotting system includes final performance summaries that highlight key metrics in bar chart format, making it easy to assess overall model quality at training completion. All visualization outputs are automatically saved as high-resolution images suitable for research documentation and publication. The training history preservation ensures that detailed performance analysis can be conducted post-training, supporting thorough evaluation of training effectiveness and identification of optimal stopping points or hyperparameter configurations for future training iterations.

5.4. Evaluation

The evaluation framework is designed to comprehensively assess the performance of the trained Mask R-CNN model on deepfake detection tasks. The system evaluates both detection accuracy, which measures the ability to correctly identify manipulated regions, and segmentation quality, which focuses on the precise delineation of altered areas across multiple test datasets. The evaluation pipeline integrates several key processes, including multi-threshold analysis to identify optimal confidence levels, comprehensive metric calculation using IoU, Dice score, and pixel accuracy, standardized COCO-format evaluation for performance assessment, and the generation of detailed visualizations that support effective interpretation of results.

The evaluation begins by loading the trained model with an architecture identical to the one used during training. This ensures consistency between training and evaluation environments. The model is constructed using a ResNet-50 backbone with a Feature Pyramid Network (FPN), and the final prediction heads for both bounding boxes and masks are reconfigured to match the number of classes in the dataset.

The system can evaluate multiple test sets simultaneously, with configurable dataset paths that support different evaluation scenarios. The full model loading implementation is presented in Appendix 8.

The evaluation framework incorporates several mask-specific metrics that are essential for segmentation quality assessment. Intersection over Union (IoU) measures the overlap between predicted and ground truth masks and is calculated as the ratio of intersection area to union area. The Dice score provides an alternative similarity measure that is particularly sensitive to smaller regions, computed as twice the intersection divided by the sum of the areas of prediction and ground truth. Pixel accuracy complements these measures by evaluating the proportion of correctly classified pixels across the entire mask.

The complete set of metric calculation functions is presented in Appendix 9.

Beyond segmentation quality, the evaluation framework also measures detection performance using widely adopted object detection metrics. Precision is defined as the ratio of correct positive predictions to all predicted positives, while recall quantifies the ratio of correct positive predictions to all actual positives. The F1-score, which is the harmonic mean of precision and recall, provides a balanced assessment of detection accuracy. The system also records the counts of true positives, false positives, and false negatives to support detailed error analysis.

An important feature of the evaluation framework is the implementation of a multi-threshold analysis strategy. Instead of evaluating the model at a fixed confidence threshold, the system explores performance across several thresholds (0.1, 0.3, 0.5, 0.7, 0.9). This approach allows the identification of optimal operating points suited to different deployment contexts, provides insights into model confidence calibration, supports the generation of precision–recall curves, and enables threshold-specific optimization.

This evaluation strategy provides a deeper understanding of model behavior across confidence levels and guides the selection of thresholds for practical deployment, also showed in Appendix 13.

Ground truth annotations stored in COCO format are processed to match the output format of the model. The pipeline includes extracting masks from polygonal annotations, mapping labels to class indices, scaling bounding boxes to the resized image dimensions, and applying quality filters to remove invalid or corrupted annotations. The complete dataset evaluation process is presented in Appendix 10.

For every dataset and threshold combination, the system computes summary statistics including precision, recall, and F1-score. These metrics are derived from aggregated counts of true positives, false positives, and false negatives. The system automatically identifies the best-performing configurations by analyzing F1-scores across all datasets and thresholds. This allows clear recommendations to be made regarding the most effective confidence thresholds for deployment. The complete visualization implementation is presented in Appendix 14.

The evaluation framework generates a variety of visualizations that assist in interpreting results. Precision–recall curves illustrate the trade-off between precision and recall across different thresholds, bar charts compare key metrics across multiple datasets, and line plots reveal how performance metrics evolve with confidence thresholds. The complete visualization implementation is presented in Appendix 11.

All evaluation results are consolidated into a structured JSON report that includes configuration details such as model paths, input dimensions, and thresholds, as well as dataset information, detailed metrics across threshold–dataset combinations, best performance recommendations, and summary statistics for rapid reference.

5.5. Model comparison

The evaluation phase of this research involves a comprehensive comparison of three distinct training approaches using different dataset subset sizes. This systematic evaluation aims to understand the relationship between dataset size and model performance, providing insights into the optimal training configuration for deepfake detection using Mask R-CNN architecture. The primary objectives of this evaluation framework encompass analyzing the impact of dataset size on model performance, comparing convergence patterns across different training configurations, evaluating computational efficiency and resource utilization, assessing class-specific detection accuracy for fake and real image segments, and determining the optimal balance between dataset size and model performance. This multi-faceted approach ensures a thorough understanding of how data scaling affects the deepfake detection system's effectiveness.

Three training configurations were implemented to assess the scalability and performance of the deepfake detection system. Subset 1000 uses 1,000 training and 100 validation images, serving as the baseline with minimal data. Subset 2500 includes 2,500 training and 300 validation images, representing intermediate scaling and highlighting improvements with moderate data growth. Subset 5000 uses 5,000 training and 800 validation images, providing a large-scale assessment to examine performance trends and potential plateaus.

The model initialization process ensures reproducible results across all configurations through careful checkpoint management and consistent random seed initialization. The full code is presented at Appendix 15 .

The evaluation uses multiple metrics to capture model performance from different angles. Primary metrics include IoU for segmentation accuracy, training/validation loss for convergence, and class-specific accuracy for fake/real detection. Secondary metrics cover convergence speed, processing time per batch, average confidence scores, and detection count statistics, together providing a holistic view of the system's behavior.

The evaluation process implements a multi-faceted approach to model assessment that goes beyond simple accuracy measurements. IoU calculation serves as the primary indicator of segmentation quality, implementing a threshold-based approach to determine overlap between predicted and ground truth masks. The implementation carefully handles edge cases where predictions or ground truth annotations are absent, ensuring robust evaluation across diverse scenarios.

Class accuracy assessment evaluates each model's ability to distinguish between fake and real image segments through class-specific accuracy metrics. This analysis provides insights into potential bias patterns or specialization tendencies that may emerge from different training dataset sizes. The full code is presented at Appendix 16.

Performance profiling incorporates processing time analysis to evaluate computational efficiency, which proves crucial for real-world deployment scenarios. This analysis considers batch processing times, memory utilization patterns, and inference speed characteristics across the different model configurations.

The evaluation framework generates twelve distinct visualization plots to provide comprehensive insights into model behavior and performance characteristics. Training dynamics plots track convergence patterns through training loss comparison, validation loss comparison, IoU progression monitoring, and overfitting analysis comparing training versus validation loss gaps.

Performance analysis plots summarize results through final performance bar charts, class-specific accuracy trends for both fake and real detection, convergence speed analysis measuring epochs required for stabilization, and training time scaling assessment of computational resource requirements.

Advanced analytics incorporate best performance summary with consolidated metric comparison, learning rate effect analysis for optimization behavior assessment, memory usage analysis for resource utilization patterns, and statistical significance testing with confidence interval determination. The visualization system employs consistent color coding and styling to facilitate interpretation across all generated plots. The full code is presented at Appendix 17.

Fake detection accuracy analysis reveals varying performance levels across dataset sizes, indicating the importance of sufficient training examples for complex fake pattern recognition. The results suggest that fake detection benefits significantly from larger training datasets, likely due to the diverse nature of deepfake generation techniques that require extensive exposure during training.

Real detection accuracy demonstrates more consistent performance across configurations, suggesting inherent differences in pattern complexity between real and fake image characteristics. This finding implies that real image features may be more fundamentally stable and easier to learn with limited data, while fake detection requires more comprehensive training examples.

The class imbalance analysis reveals how different dataset sizes affect the model's ability to maintain balanced performance between fake and real detection tasks. Larger datasets tend to provide more stable performance across both classes, while smaller datasets may exhibit bias toward one class depending on the specific data distribution characteristics.

An automated reporting system generates detailed comparison tables and statistical analyses that complement the visual representations. The system creates structured comparison tables containing model names, dataset sizes, performance metrics, training durations, and accuracy measurements for systematic analysis.

The reporting framework incorporates statistical analysis capabilities that identify best-performing configurations, calculate improvement ratios between different dataset sizes, assess efficiency metrics comparing performance gains against computational costs, and generate insights regarding scaling behavior patterns. This automated approach ensures consistent reporting format and eliminates potential manual analysis errors. The full code is available at Appendix 18.

6. Experimental results

6.1. Dataset definition

The dataset is the foundation of any machine learning system since it is the primary source from which models extract significant patterns. A consistent and dependable approach to model training and evaluation is made possible by a well-structured dataset. Image segmentation tasks require extensive annotations, including pixel-level masks, to facilitate the learning of fine-grained spatial features. This is especially important in sensitive fields like deepfake detection.

6.1.1. Dataset selection

A specialized dataset was used, containing annotated facial images for the task of image forgery segmentation. The dataset is structured in the COCO format and includes high-resolution visual data with pixel-level segmentation masks identifying both manipulated (fake) and authentic (real) facial regions. Each annotation specifies the bounding box and the corresponding class label: 1 for fake and 2 for real, while 0 is reserved for the background [4].

The dataset is divided into two main parts: training and validation. The training set contains 41864 images, while the validation set consists of 800 images. All images were resized to a consistent resolution of 256×256 pixels to balance the trade-off between detail preservation and computational efficiency, especially given the constraints of a 4GB VRAM GPU used during training.

Each image may include multiple face instances, with corresponding masks and labels: fake (class 1) or real (class 2). This structure allows the segmentation model to learn from a wide variety of real-world scenarios, including different lighting conditions, facial expressions, backgrounds, and levels of visual complexity. The dataset includes both indoor and outdoor scenes, occlusions, and varying facial orientations to ensure the robustness of the training process. This dataset was selected due to its rich annotation format, multi-face support, and visual diversity, which are essential for training a reliable deepfake segmentation model capable of generalizing to realistic and unpredictable input data.

The dataset also includes two additional subsets for testing: Test-Dev and Test-Challenge. The Test-Dev set contains a separate collection of unseen images that were not used during training or validation. These images serve as a controlled test set to evaluate how well the model generalizes to new, real-world cases. The images vary in terms of background, facial positioning, lighting, and subject diversity, providing a reliable benchmark for measuring the model's ability to detect real and fake faces outside the training distribution.

The Test-Challenge subset introduces a more difficult scenario. It contains augmented versions of test images that have been deliberately modified through various transformations such as blurring, color distortions, occlusions, and other visual perturbations. This set is designed to test the model's robustness and its ability to identify forgeries under degraded or altered visual conditions. The goal is to assess whether the segmentation model can still distinguish between manipulated and authentic faces even when image quality is reduced or when common deepfake disguises are applied.

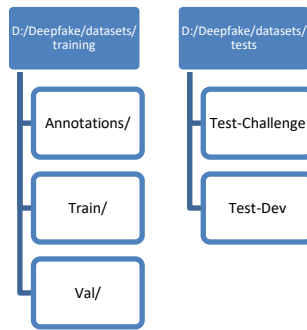


Figure 6.1. Dataset structure

Table 6.1 presents two sample images per class (fake and real) taken from the training and validation sets. The images in the fake class represent manipulated facial regions generated through deepfake techniques, often showing artifacts such as blending inconsistencies, unnatural expressions, or irregular skin textures.

Table 6.1. Example Images

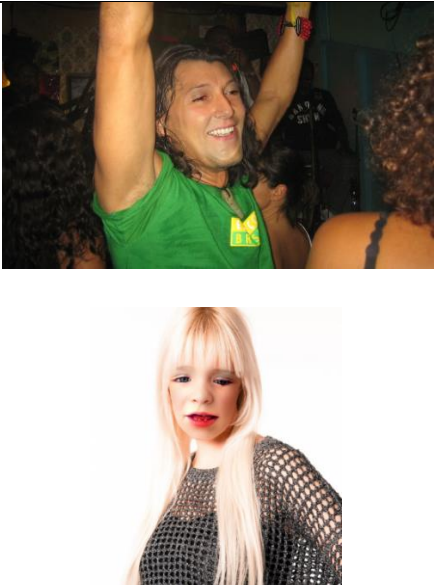
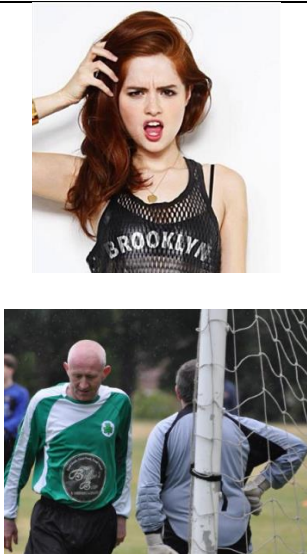
	Fake	Real
Dataset for training		

Table 6.2 presents two sample images per class, selected from the Test-Dev and Test-Challenge sets. These subsets were not used during training or validation, making them ideal for evaluating the model's ability to generalize. The Test-Dev examples represent standard real-world conditions, including varying backgrounds, multiple faces, and natural lighting. These samples test the model's consistency and segmentation performance on clean, unseen data. The Test-Challenge images were deliberately modified to include visual transformations such as blurring, colour distortion, and occlusion. These augmented samples are designed to simulate adversarial conditions and evaluate the model's robustness when image quality is compromised.

Table 6.2. Test examples

Dataset	Fake	Real
---------	------	------

Test-Dev		
Test-Challenge		

6.1.2. Label mapping

Each image was annotated with structured pixel-level masks and bounding boxes that corresponded to class labels as part of the supervised learning process used to train the segmentation model. Each identified face instance is given a `category_id` to indicate whether the region is real (authentic) or fake (manipulated), and the annotations adhere to the COCO format. The model treats the background as class 0 implicitly rather than explicitly labelling it. For this project, three classes were defined:

- Label 0 – Background (non-face regions)
- Label 1 – Fake (forged or deepfake face regions)
- Label 2 – Real (authentic, unaltered face regions)

Table 6.3 Example label mapping for training

Image ID	Filename
1	D:\Deepfake\datasets\Train\44ef213aa.jpg
2	D:\Deepfake\datasets\Train\8ded94d4df.jpg
3	D:\Deepfake\datasets\Train\85ce8a2508.jpg
11	D:\Deepfake\datasets\Train\6d92e3a6f4.jpg
22	D:\Deepfake\datasets\Train\17091a2a6a.jpg
33	D:\Deepfake\datasets\Train\3e13fa971c.jpg
40	D:\Deepfake\datasets\Train\3da5bd35d1.jpg

Table 6.4 Example label mapping for testing

Image ID	Filename
1	D:\Deepfake\datasets\Test-Dev\f37e15d108.jpg
2	D:\Deepfake\datasets\Test-Dev\5f121a747b.jpg
10	D:\Deepfake\datasets\Test-Dev\caf1f7c59a.jpg

11	D:\Deepfake\datasets\Test-Dev\0d5724d2ec.jpg
20	D:\Deepfake\datasets\Test-Dev\161e417c7c.jpg
22	D:\Deepfake\datasets\Test-Dev\0a600dbfab.jpg
25	D:\Deepfake\datasets\Test-Dev\ccd986aa78.jpg

Table 6.3 presents a sample mapping of image IDs and filenames from the training set, showing how each image is referenced in the dataset. Table 6.4 provides a similar mapping for the testing set, listing selected image IDs and corresponding file paths used for model evaluation.

6.2. Optimization

Several useful optimization techniques were used to increase training stability and efficiency to meet the high computational needs of deepfake segmentation on constrained hardware. It is difficult to train a complicated model like Mask R-CNN because the system was built to run on an NVIDIA GTX 1650 Ti GPU with 4GB of VRAM. The model was initially trained on a restricted selection of 1,000 training photos and 100 validation images rather than the entire dataset. After then, the size of the dataset was progressively expanded to include 2,500 training and 300 validation samples, and ultimately, 5,000 training and 800 validation photos. With the aid of this progressive method, the model was able to train rapidly with minimal resources and steadily get better as more data was added.

The AdamW optimizer, which isolates weight decay from gradient updates, was used for training. Better regularization is facilitated by this, particularly when training on noisy or tiny datasets. A mixed loss function comprising mask loss, bounding box loss, and classification loss was used to train the model. These made it easier for the model to identify, locate, and precisely segment phony sections.

A ReduceLROnPlateau scheduler was utilized to better adapt learning during training. If the validation loss did not decrease for three consecutive epochs, this tool cut the learning rate in half. As a result, the model was able to progress from learning rapidly at first to producing smaller, more accurate updates later.

To control VRAM utilization, extra measures were required because of the GPU's limited memory. To keep the GPU cache from running out of memory, `torch.cuda.empty_cache()` was used to manually remove it every 20 batches. Additionally, during training, memory utilization was monitored in order to spot problems early. To maintain training stability, this assisted in directing changes like lowering the batch size or image resolution.

6.3. Parameters sets comparison

The project was implemented using Python 3.10 within a virtual environment (venv) managed through PyCharm. This approach provided an isolated and reproducible workspace, which was essential given the complex interplay between computer vision libraries, dataset handling tools, and GPU-based training workflows reliant on CUDA support.

Development was conducted using the PyTorch framework, complemented by torchvision utilities for tasks such as image transformation, data augmentation, and loading pretrained backbone models. The primary segmentation model employed was Mask R-CNN, adapted to handle multi-class mask predictions and integrated with the COCO evaluation framework. The system was designed to support all stages of experimentation, including training, inference, visualization, and evaluation of real versus fake face regions in manipulated images.

The core task involved binary segmentation, classifying each pixel into either a real or fake region. Training leveraged the existing polygon mask annotations that comply with the COCO 2017 dataset structure established in the initial dataset preparation phase. During preprocessing, images underwent dynamic resizing and padding to achieve optimal input dimensions for the network architecture, with resolution adjustments made based on available computational resources and model requirements.

To examine how training dynamics respond to hyperparameter adjustments, two separate training configurations were tested. These differed in parameters such as learning rate, batch size, weight decay, checkpoint interval, and early stopping patience. This dual-setup strategy was instrumental in evaluating training stability, model convergence, and the ability to generalize across varied forgery patterns.

Table 6.5 Variation of training parameters

Parameters	Set 1	Set 2
Epochs	15	15
Learning rate	1×10^{-4}	5×10^{-6}
Batch size	4	2
Workers	2	4
Scheduler	True	True
Weight_decay	5×10^{-6}	0
Checkpoint_freq	2	10
Early_stopping_patience	4	8

Table 6.5 compares two training setups, set 1 and set 2, used to test how different hyperparameters affect the performance of the deepfake segmentation model. These configurations were chosen to explore how changes in training settings influence the model's ability to learn effectively, remain stable during training, and perform well on new, unseen data. Each parameter plays an important role in helping the model learn to detect fake facial regions in challenging image datasets.

In both setups, the number of training epochs was fixed at 15. Keeping the number of epochs the same ensured that any differences in performance came from other settings rather than how long the model was trained. One epoch means the model has seen the entire training dataset once.

A major difference between the two configurations was the learning rate. Set 1 used a higher learning rate of 1×10^{-4} , which helps the model learn faster but can sometimes lead to unstable training. Set 2 used a smaller learning rate of 5×10^{-6} , allowing the model to learn more slowly and steadily. This is especially useful when working with noisy data or when fine-tuning a pretrained model.

The batch size, which defines how many samples are processed at once before updating the model's weights, was also different. Set 1 used a batch size of 4, which balances speed and memory usage. Set 2 used a smaller batch size of 2, which can lead to more variation during training but may help the model better learn detailed patterns, especially when working with large or high-resolution images.

The number of data loader workers, which handle loading data in parallel during training, also varied. Set 1 used 2 workers, while Set 2 used 4, improving the speed of data loading. This is helpful when working with large datasets that include many high-resolution images.

Both setups used a scheduler to automatically reduce the learning rate during training when the model stops improving. This helps the model learn more precisely and avoid getting stuck at poor solutions.

Weight decay was used in Set 1 with a value of 5×10^{-6} to help prevent the model from overfitting by limiting how large the weights can grow. Set 2 did not use weight decay, instead relying on a lower learning rate and early stopping to maintain generalization.

The checkpoint interval, which defines how often the model's progress is saved, was every 2 epochs in Set 1 and every 10 epochs in Set 2. More frequent checkpoints in Set 1 allow for easier recovery and tracking, while Set 2 reduces the number of saved files but gives fewer updates during training.

Early stopping patience was set to 4 epochs in Set 1 and 8 in Set 2. This means Set 1 stops training sooner if there is no improvement, while Set 2 allows more time for the model to improve, especially when progress is slower or unstable.

6.3.1. Set results comparison

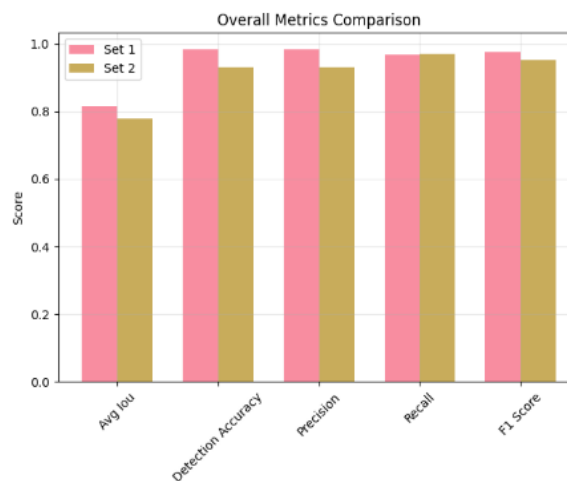


Figure 6.2 Overall Metrics Comparison

Set 1 achieves a detection accuracy of 98.31%, significantly outperforming Set 2's 93.10% by a margin of 5.21 percentage points. The precision values mirror this pattern exactly, with Set 1 maintaining 98.31% precision compared to Set 2's 93.10%. For the F1-score, Set 1 records 97.49% versus Set 2's 95.08%, representing a 2.41 percentage point advantage. However, Set 2 demonstrates marginally superior recall performance at 97.14% compared to Set 1's 96.69%, though this difference of only 0.45 percentage points is negligible. The average IoU metric shows Set 1 achieving 81.62% localization accuracy against Set 2's 77.82%, indicating a 3.80 percentage point improvement in object boundary detection precision.

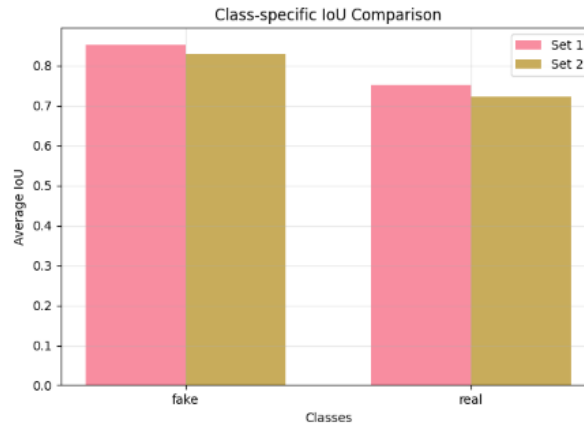


Figure 6.3 Class-specific IoU Comparison

The class-specific analysis reveals distinct performance patterns between fake and real content detection. For fake content detection, Set 1 achieves an average IoU of 85.18% with 760 predictions, while Set 2 attains 83.00% IoU across 699 predictions, demonstrating Set 1's 2.18 percentage point advantage in fake content localization. The performance gap widens significantly for real content detection, where Set 1 records 75.29% IoU compared to Set 2's 72.25% IoU, representing a 3.04 percentage point difference. Both models exhibit weaker performance on real content detection, with IoU values approximately 10 percentage points lower than their fake detection capabilities, suggesting that authentic content presents greater localization challenges than manipulated content.

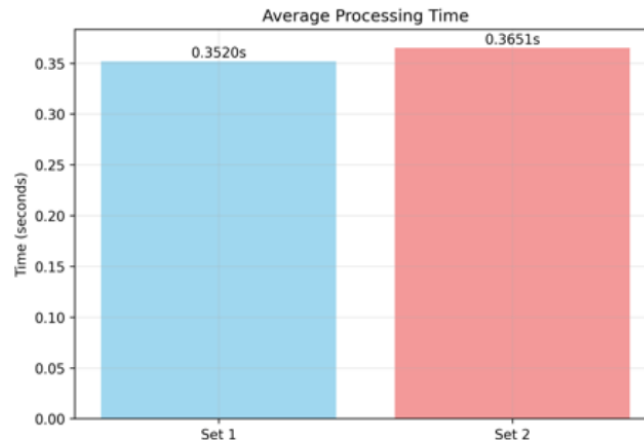


Figure 6.4 Average Processing Time

The processing efficiency comparison shows Set 1 completing inference in an average of 0.3520 seconds per sample, while Set 2 requires 0.3651 seconds, resulting in a 0.0131-second advantage for Set 1. This represents a 3.6% speed improvement, which translates to meaningful efficiency gains when processing large datasets. Both models demonstrate sub-second processing capabilities suitable for real-time applications, with Set 1's faster inference time providing additional operational advantages without compromising accuracy performance.

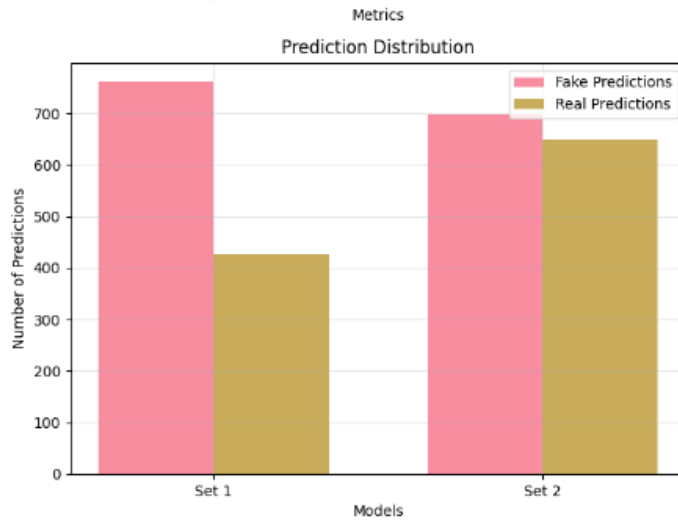


Figure 6.5 Prediction Distribution

The stacked bar chart reveals significant differences in prediction behavior between the two models. Set 1 generates 1,187 total predictions (760 fake, 427 real), while Set 2 produces 1,348 predictions (699 fake, 649 real), indicating Set 2's 13.6% higher prediction volume. Set 1 exhibits a 1.78:1 fake-to-real prediction ratio, whereas Set 2 shows a more balanced 1.08:1 ratio. This suggests Set 1 adopts a more conservative prediction strategy, potentially contributing to its higher precision, while Set 2's aggressive prediction approach may explain its marginally higher recall but lower precision performance.

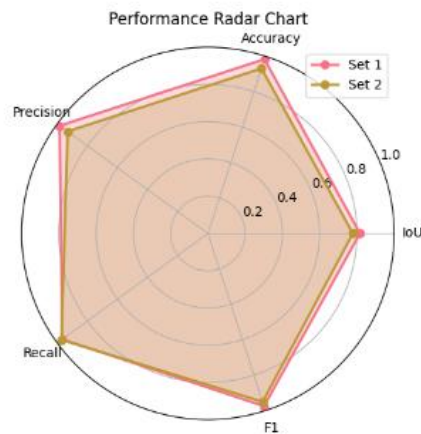


Figure 6.6 Performance Radar Chart

The radar chart visualization clearly illustrates Set 1's comprehensive performance superiority through its larger coverage area across all metrics except recall. Set 1 forms a more regular pentagon shape with consistently high values approaching the outer boundary (1.0) for accuracy, precision, IoU, and F1-score. Set 2's profile shows a smaller overall area with a single protrusion in the recall dimension, where it marginally exceeds Set 1's performance. The overlapping regions occur primarily in recall, while Set 1 maintains clear advantages in all other dimensions. The visual area

comparison demonstrates Set 1's balanced excellence across multiple performance criteria, making it more suitable for general-purpose deepfake detection applications.

The evaluation of five performance visualizations clearly identifies Set 1 as the better configuration for deepfake detection, making it the most suitable choice for the upcoming subset training experiments. The results consistently show that Set 1's training setup, 15 epochs, a learning rate of 1×10^{-4} , and weight decay of 5×10^{-6} —leads to a more accurate and dependable model compared to Set 2, which was trained with just 7 epochs, a very low learning rate (5×10^{-6}), and no weight decay.

Three key findings highlight why Set 1 performs better. First, it delivers not just faster processing, but also more precise predictions, making fewer unnecessary detections and reducing false positives. This is especially important for tasks where accuracy is critical. Second, Set 1 shows strong, consistent performance across both real and fake content, which suggests that its longer training time and regularization help it learn the key differences between genuine and manipulated images more effectively. Third, the radar chart visualization confirms that Set 1 performs well across all major evaluation metrics, rather than excelling in only one area, indicating a well-balanced and stable model.

Analyzing the training parameters further helps explain these results. Set 1's combination of a higher learning rate and longer training duration allows the model to learn more meaningful features, while the added weight decay reduces overfitting, improving performance on new, unseen data. In contrast, Set 2's low learning rate and shorter training period likely lead to underfitting, preventing the model from fully learning the complex patterns typical of deepfake manipulations.

As a result, Set 1's configuration will be used as the standard model setup for the following subset training experiments using datasets of 1000/100, 2500/300, and 5000/800 samples for training and validation. This ensures that all future comparisons on dataset size will be made using the most effective baseline model, providing consistent and meaningful insights into how dataset volume impacts deepfake detection performance.

6.4. Subsets comparison results

Training deep learning models for deepfake detection presents both computational and design challenges, especially when working with large datasets exceeding 40,000 images. To address this, a progressive subset training strategy was adopted, starting with 1,000 training samples and gradually increasing to 2,500 and 5,000, each with corresponding validation sets. This approach enabled efficient testing of model configurations, optimization routines, and training dynamics without the full computational cost of large-scale training from the outset.

Faster experimentation was made possible by smaller subsets, which aided in the improvement of preprocessing procedures, loss functions, and architectures. The overall robustness of the model was improved by this iterative method, which made it simpler to spot overfitting and generalization problems early. The subset technique demonstrated how performance increased with data size and indicated the threshold at which the benefits of adding more samples were minimal. This phased approach exposes the model to progressively more complicated learning techniques that are mirrored in the curriculum. A slow learning route like this can result in more stable convergence and better feature representation in deepfake detection, where minor visual cues are commonly used to distinguish between real and fake. To provide a basis for full-scale training on the entire dataset, this chapter shows how scaling from 1,000 to 5,000 images promotes both methodological rigor and practical efficiency.



Figure 6.7 Training and validation performances for the all the dataset used

Figure 6.7 provides an in-depth comparative assessment of three Mask R-CNN models trained on datasets of different sizes, 1000, 2500, and 5000 images, targeted at the task of deepfake segmentation. The figure is structured into three aligned graphs: the first (left) tracks the trajectory of training loss over epochs, the second (center) shows the validation loss trend, and the third (right) illustrates the development of the Intersection over Union (IoU) score on the validation dataset. Taken together, these visualizations demonstrate how increasing the quantity of training data influences the model's learning process, generalization capacity, and segmentation performance.

The plot of training loss (left) confirms that all three models undergo effective optimization, as reflected by the steady downward trends across epochs. The model trained with 5000 images achieves the most favourable outcome, ending with the lowest and most stable training loss near 0.172, indicating successful convergence and minimal risk of underfitting. The 2500- and 1000-image models also show appropriate learning patterns, though the 1000-sample variant exhibits a temporary spike around epoch 13. This fluctuation could be attributed to the variability inherent in small datasets or to the influence of noisy batches. Overall, the results suggest that expanding the training dataset contributes to smoother convergence and more reliable optimization.

In the validation loss plot (center), the patterns are more complex. Although the 5000-image model eventually produces the lowest validation loss, around 0.317, it does not maintain the most consistent loss trend throughout training. The 2500-image model, by comparison, sustains a relatively stable and low validation loss over time, finishing with a final value of approximately 0.258. The 1000-image model, on the other hand, shows a clear upward trend in validation loss beyond epoch 5, ending at approximately 0.460. This increase may signal overfitting, likely due to the model's limited exposure to diverse training data or insufficient regularization. These outcomes suggest that while a larger dataset generally supports better generalization, effective validation procedures and training strategies are also critical.

The intersection over union (IoU) score comparison (right) offers a direct evaluation of segmentation accuracy. The model trained with 5000 images outperforms the others, achieving a peak and stable final IoU of around 0.83. The 2500-image model follows closely with a final IoU of roughly 0.77, while the 1000-image model concludes slightly behind at 0.76. The model trained on the smallest dataset shows some decline in IoU mid-training, consistent with its increasing validation loss. The close performance between the 1000- and 2500-image models could stem from variability in the validation data or from imbalanced class distributions. Interestingly, the 2500-image model experiences a final-epoch jump in IoU, which might reflect an incidental alignment with the validation set. Further testing or cross-validation would be needed to confirm this behavior.



Figure 6.8 Evaluation : intersection over union (IoU), class-wise metrics, and confusion matrix for first subset

Figure 6.8 presents a comprehensive evaluation of the trained Mask R-CNN model applied to the deepfake segmentation on subset of 1000 annotated images. The figure consists of three parts: the IoU score distribution (left), the class-wise performance metrics (center), and the confusion matrix (right), each providing insights into the model's predictive quality and class discrimination capability.

The IoU score distribution shows how well the predicted segmentation masks overlap with the ground truth across all test samples. The model achieves an average IoU of 0.702, indicating that, on average, 70% of the object area is correctly segmented. A substantial cluster of samples reaches high IoU values (above 0.75), suggesting that the model performs reliably on a significant portion of the dataset. However, the left-skewed tail of the histogram reveals a notable number of samples with IoU near zero. These indicate complete failure to predict any meaningful overlap for certain instances. Such failures likely stem from challenging image conditions, occlusions, or class imbalance and highlight the model's limited generalization under less favorable conditions.

The middle chart illustrates the class-wise precision, recall, and F1-score for the semantic labels “fake” and “real.” The fake class shows outstanding results, with a precision of 0.96 and a recall of 1.00, yielding an F1-score of 0.98. This demonstrates the model's high sensitivity and specificity for detecting manipulated facial regions. In contrast, the real class suffers from a substantial drop in recall, achieving only 0.63, with a corresponding F1-score of 0.755. This discrepancy suggests that the model often mislabels real images as fake, likely due to either the predominance of fake instances in the dataset or a lack of strongly distinguishing features in real content. The imbalance between the two classes indicates a detection bias toward synthetic forgeries.

The confusion matrix on the right confirms this interpretation. Out of 500 test samples, the model correctly identifies 470 fake instances and misclassifies only one. However, only 9 real samples are correctly labelled, while 20 are erroneously predicted as fake. This skewed classification pattern highlights the model's limited ability to capture real facial features with confidence. It reflects a conservative decision boundary, wherein the model heavily favors the “fake” label when uncertain. Such a bias could have practical implications, particularly in real-world applications where false positives misclassifying authentic content as fake can undermine trust or trigger unjustified consequences.



Figure 6.9 Evaluation : intersection over union (IoU), class-wise metrics, and confusion matrix for the second subset

Figure 6.9 presents the results from a subsequent evaluation of the Mask R-CNN model trained on a subset of 2500 annotated images, reflecting noticeable improvements in both segmentation consistency and class-wise discrimination. Although the figure maintains the same structural components IoU distribution (left), class-wise performance chart (center), and confusion matrix (right) this experiment highlights how expanding the training dataset beyond 1000 images leads to better generalization and more balanced performance across semantic classes.

In this updated experiment, the model achieves a mean IoU of 0.688, which is slightly below the earlier 0.702 but accompanied by a more cohesive distribution. Most predictions remain tightly clustered around higher IoU values, particularly above 0.75, with fewer instances falling near zero. This suggests a reduction in segmentation failures, indicating that training with 2500 images has helped the model handle more diverse visual variations and improved its resilience on edge cases.

A significant enhancement is observed in the real class detection. The model now achieves a recall of 0.73 and an F1-score of 0.80, compared to 0.63 and 0.755 respectively in the earlier evaluation based on only 1000 training samples. Precision for the real class also improves to 0.89, meaning the model is now much better at correctly identifying authentic facial regions without over-predicting fakes. Meanwhile, performance on the fake class remains strong, with a precision of 0.96, recall of 0.98, and F1-score of 0.97, demonstrating that the expansion in training data did not compromise the model's ability to detect manipulated content. These improvements reflect a significant reduction in class imbalance effects, suggesting that the larger dataset has allowed the model to better learn from subtle differences between real and fake instances.

The confusion matrix (right) reinforces these trends. Among 500 test samples, the model correctly identifies 10 real images, an increase from the previous 9, and reduces false negatives from 20 to 17. Only 4 fake images are misclassified, compared to 1 previously, which is still within a low and acceptable margin given the overall accuracy. The distribution shows that the model has adopted a more balanced decision-making process, becoming less reliant on over-predicting the “fake” label in ambiguous cases.



Figure 6.10 Evaluation : intersection over union, class-wise metrics, and confusion matrix for last subset

Figure 6.10 presents the evaluation results of the Mask R-CNN model trained on a larger subset of 5000 annotated images, showing substantial improvements in both segmentation accuracy and class-level balance. The figure follows the established three-panel format: the IoU score distribution on the left, class-wise performance metrics in the center, and the confusion matrix on the right. This expanded training configuration contributes to stronger generalization and more reliable classification of both real and manipulated facial regions.

In this evaluation, the model achieves a mean IoU of 0.767, representing a notable increase in segmentation accuracy compared to prior runs. Most predictions cluster tightly around high IoU values, particularly above 0.75 with very few falling near zero, suggesting a clear reduction in failure cases. The improved distribution indicates more consistent mask generation and a higher degree of spatial alignment with ground truth annotations. These gains are likely the result of both increased data diversity and refined training strategies applied during model development.

A marked advancement is observed in the detection of real images. The model now records a recall of 0.94 and an F1-score of 0.96, a significant improvement over earlier evaluations. Precision also rises to 0.98, indicating that the model rarely misclassifies fake content as real. At the same time, performance on the fake class remains robust, with precision at 0.98, recall at 0.99, and an F1-score of 0.99. These metrics demonstrate that expanding the training set has not only preserved the model’s effectiveness in identifying manipulated content but also resolved previous class imbalance issues. The model is now capable of reliably recognizing authentic images without increasing the false acceptance rate of deepfakes.

The updated confusion matrix further validates these outcomes. Of the 500 test samples, the model correctly classifies 487 real images, reducing the number of real-to-fake misclassifications to just 13. Meanwhile, only 2 fake samples are incorrectly labelled, reinforcing the model’s high precision for fake detection. This improved distribution suggests that the model’s decision boundary has become more calibrated, effectively minimizing its prior tendency to default to the “fake” label when uncertain. As a result, classification is now more balanced, with stronger confidence in both semantic categories.

6.5. Visual evaluation

The segmentation results from the three Mask R-CNN models trained on subsets photos, are visually compared in this section to further highlight the effect of training data volume on model performance. To subjectively evaluate the growth in segmentation accuracy and class prediction reliability with increasing dataset size, representative test samples and their related predicted masks

are presented. The quantitative measurements previously presented are supplemented by these side-by-side visualizations, which show gains in class discrimination, mask completeness, and boundary precision over different training sizes.

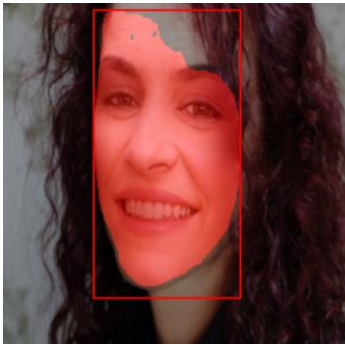
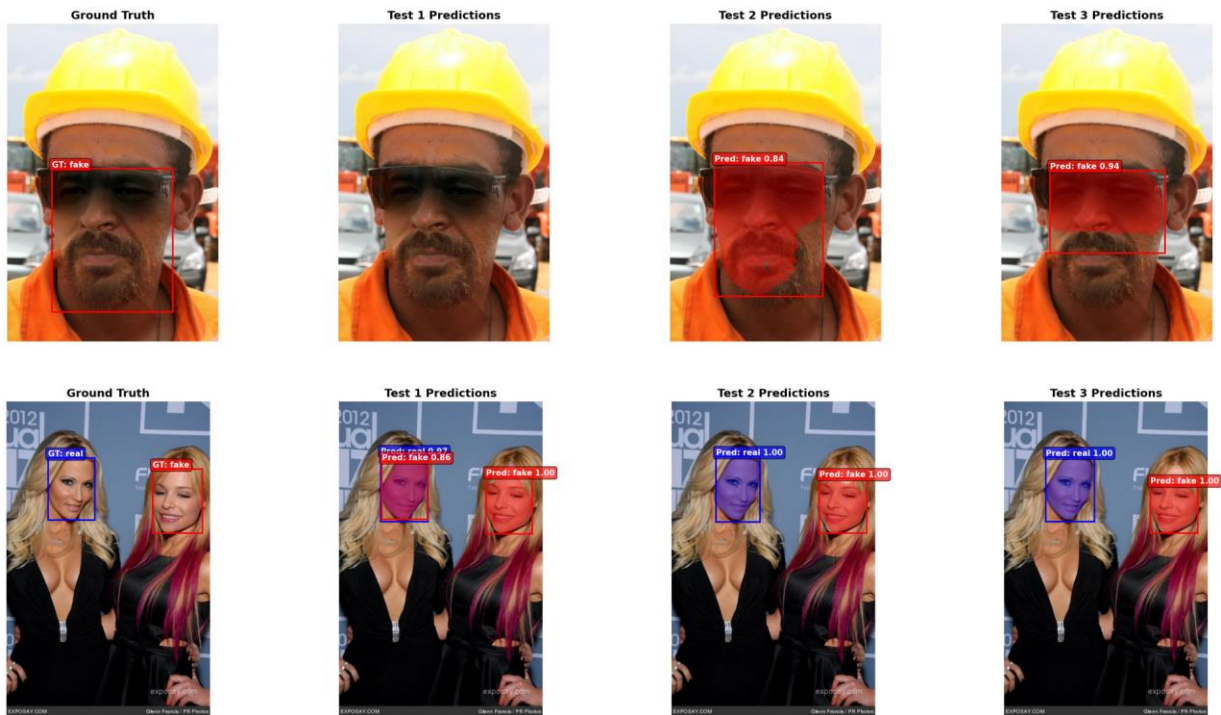


Figure 6.11 Model prediction

The figure 6.11 represents a test image where the predicted segmentation mask is overlaid on the face of the subject. The highlighted region indicates the area detected by the model as potentially manipulated. In every test image, the model creates a segmentation mask that covers areas of the face that might have been altered. By progressively improving these masks across several epochs of training, the Mask R-CNN modifies its predictions in response to ground truth annotations. The ability of the model to recognize minor visual cues, which frequently signal tampering, such as blending artifacts, uneven textures, or unnatural edges, increases with training and the number of cases it is exposed to. As time goes on, this allows the model to more precisely identify real and false face content in addition to precisely localizing the modified areas.



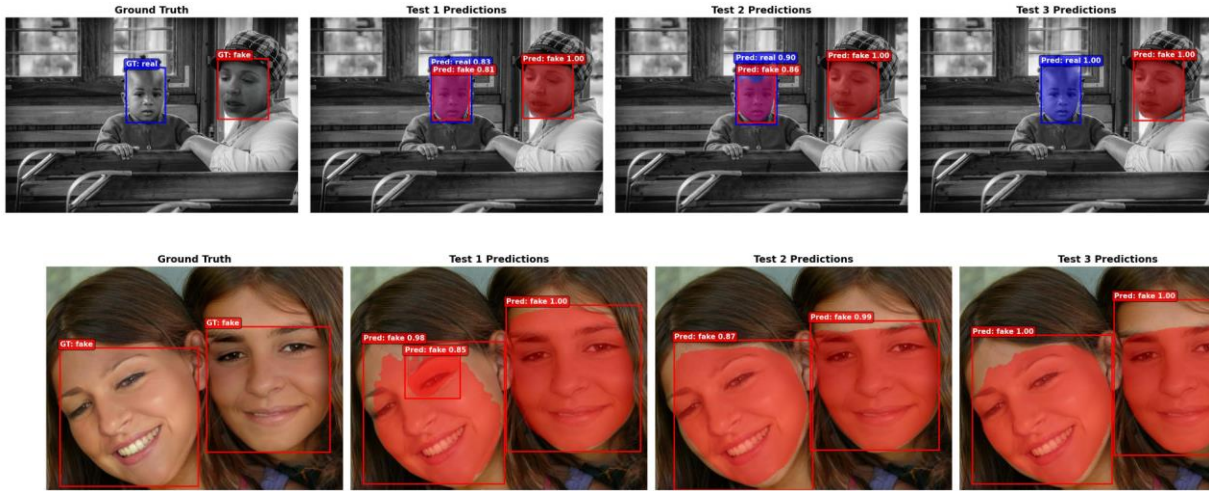


Figure 6.12: Comparison of predicted masks from models trained on three different dataset subsets (subfigures a–d)

To demonstrate how different training configurations influence performance, three prediction outputs are shown, each produced by a model trained on a distinct subset of the dataset. As illustrated in Figure 6.12(d), models trained on larger and more diverse datasets consistently generate sharper, more precise segmentation masks with cleaner boundaries and fewer false detections. In contrast, models trained on limited data are more likely to miss manipulated regions or over-segment, producing unnecessary mask areas. This reinforces the critical role of data quality and variety in enabling the Mask R-CNN to identify forgeries effectively. With richer training subsets, the model becomes more adept at detecting subtle tampering cues such as irregular textures or unnatural boundary transitions, ultimately yielding superior segmentation outcomes.

For instance, in Figure 6.12(a), only a small area around the subject’s eyes has been altered. The final model prediction accurately isolates this region without extending the mask across the entire face, showing refined spatial precision. By comparison, earlier predictions in Figures 6.12(b) and 6.12(c) highlight inconsistent or incorrect areas, sometimes labeling the same region as both real and fake. This uncertainty is eliminated in the final model’s output, which clearly distinguishes between tampered and authentic portions. These results underscore how training on extensive, well-structured data enhances the model’s discrimination capability, enabling it to produce segmentation masks that are both accurate and semantically correct, even in complex or subtly manipulated scenarios.

7. Conclusion

This study investigated the design and evaluation of a deepfake image segmentation system leveraging Mask R-CNN, applied to a COCO-format dataset featuring detailed annotations of both genuine and manipulated facial regions. The primary objective was binary semantic segmentation classifying each pixel as either "real" or "fake" with an emphasis on achieving high-accuracy mask predictions under varying and complex conditions.

Due to hardware constraints, specifically a 4GB VRAM GPU, a staged training approach was implemented. The model was trained using three progressively larger data subsets (1,000, 2,500, and 5,000 images), which enabled gradual improvement of training techniques and parameter tuning. Two training configurations were tested, and the configuration referred to as Set 1, defined by a learning rate of 1×10^{-4} , batch size of 4, and weight decay of 5×10^{-6} , consistently outperformed Set 2 across evaluation metrics. Set 1 achieved notable results, including 98.31% accuracy, an F1-score of 97.49%, and a mean IoU of 81.62%, whereas Set 2 lagged behind with 93.10% accuracy and a mean IoU of 77.82%.

Comparative analysis of the training subsets demonstrated that increasing the dataset size positively impacted generalization and segmentation precision. Although the model trained on just 1,000 samples delivered reasonable performance (mean IoU of 0.702), it displayed a tendency to overpredict fake regions, leading to suboptimal results on authentic areas. Expanding the training set to 2,500 samples improved the model's ability to detect real content, raising the F1-score for the real class to 0.80. At 5,000 samples, segmentation performance further advanced, reaching a mean IoU of 0.767 with balanced F1-scores of 0.96 (real) and 0.99 (fake).

Qualitative assessments also revealed that larger training datasets resulted in clearer, more accurate segmentation masks, even in cases involving slight alterations or occlusions. The final model not only achieved high spatial precision but also demonstrated sensitivity to subtle visual anomalies, effectively minimizing both false positives and false negatives.

In summary, the study confirms that a progressively trained Mask R-CNN model, supported by well-tuned hyperparameters, can accurately segment deepfake facial areas. The results underline the significance of dataset scale and diversity, as well as the importance of a carefully planned training strategy, in achieving robust, real-world performance. These conclusions advocate for the continued application of incremental training workflows and optimized configurations in the field of deepfake forensics.

Generative AI tools were utilized ethically throughout the project to support code development, automate training processes, and assist with documentation. All generated content was thoroughly reviewed to ensure academic rigor and technical correctness.

Appendix

Appendix 1 - COCO JSON structure

```
{
  "categories": [
    {
      "id": 0,
      "name": "Real"
    },
    {
      "id": 1,
      "name": "Fake"
    }
  ],
  "images": [
    {
      "id": 0,
      "file_name": "9c541c578f.jpg",
      "width": 1024,
      "height": 847
    },
    {
      "id": 1,
      "file_name": "e44ef213aa.jpg",
      "width": 1024,
      "height": 683
    },
    {
      "id": 2,
      "file_name": "8ded94d4df.jpg",
      "width": 1024,
      "height": 768
    }
  ],
  "id": 13695,
  "image_id": 6065,
  "iscrowd": 0,
  "area": 606,
  "category_id": 0,
  "bbox": [
    454,
    214,
    48,
    65
  ],
  "segmentation": [
    [
      477,
      250,
      477,
      252,
      481,
      260,
      483,
      261,
      482,
      255,
      479,
      253,
      479,
      251
    ]
  ]
}
```

Figure 1: Illustrates the COCO JSON structure and its adaptation to our labeling system

Appendix 2: Dataset Initialization and Path Verification

```
print("\n Verifying paths...")
required_paths = [args.train_img_dir, args.train_ann_file, args.val_img_dir, args.val_ann_file]
for path in required_paths:
    print(f"{path} - Exists: {os.path.exists(path)}")
print("\n Creating datasets...")
train_ds = CocoDataset(args.train_img_dir, args.train_ann_file, resize_size=(256, 256), max_samples=5000)
val_ds = CocoDataset(args.val_img_dir, args.val_ann_file, resize_size=(256, 256), max_samples=800)

train_loader = DataLoader(
    train_ds, batch_size=args.batch_size, shuffle=True,
    collate_fn=collate_fn, num_workers=args.workers, pin_memory=True
)
val_loader = DataLoader(
    val_ds, batch_size=args.batch_size, shuffle=False,
    collate_fn=collate_fn, num_workers=args.workers, pin_memory=True
)

print(f"\nDataset stats:")
print(f"Training images: {len(train_ds)}")
print(f"Validation images: {len(val_ds)}")
```

Appendix 3: Memory-Optimized Training Loop with Mixed Precision

```
if torch.cuda.is_available() and batch_idx == 0:
    torch.cuda.empty_cache()
    if hasattr(torch.cuda, 'memory_allocated'):
        mem_allocated = torch.cuda.memory_allocated(0) / 1024 ** 3
        if mem_allocated > 3.5: # 3.5GB threshold for 4GB GPU
            print(f" High GPU memory usage: {mem_allocated:.2f} GB")
images = [img.to(device) for img in images]
targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
try:
    with torch.cuda.amp.autocast():
        loss_dict = model(images, targets)
        losses = sum(loss for loss in loss_dict.values())
    optimizer.zero_grad()
    scaler.scale(losses).backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    scaler.step(optimizer)
    scaler.update()
    epoch_train_loss += losses.item()
    num_batches += 1
except RuntimeError as e:
    if "out of memory" in str(e):
        print(f" GPU Out of Memory at batch {batch_idx + 1}!")
        print(" Try reducing batch_size to 2 or image_size to 256x256")
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
    raise e
```

Appendix 4: Optimizer Configuration and Learning Rate Scheduling

```
optimizer = optim.AdamW(model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
if args.use_scheduler:
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', factor=0.5, patience=3, verbose=True
    )
args = argparse.Namespace(
    epochs=15,
    lr=1e-4, # Extremely low LR
    batch_size=4,
    workers=2,
    use_scheduler=True,
    weight_decay=5e-4,
    checkpoint_freq=2,
    early_stopping_patience=4
)
if args.use_scheduler:
    scheduler.step(avg_val_loss)
```

Appendix 5: Comprehensive Evaluation Metrics Implementation

```
def calculate_iou(pred_masks, target_masks, threshold=0.5):
    """Calculate IoU between predicted and target masks"""
    if len(pred_masks) == 0 or len(target_masks) == 0:
        return 0.0
    ious = []
    pred_masks = (pred_masks > threshold).float()
    target_masks = target_masks.float()

    for pred_mask in pred_masks:
        best_iou = 0.0
        for target_mask in target_masks:
            intersection = (pred_mask * target_mask).sum()
            union = pred_mask.sum() + target_mask.sum() - intersection
            if union > 0:
                iou = intersection / union
                best_iou = max(best_iou, iou.item())
        ious.append(best_iou)
    return np.mean(ious) if ious else 0.0

def evaluate_model(model, val_loader, device):
    """Comprehensive model evaluation"""
    model.eval()
    total_iou = 0.0
    total_samples = 0
    class_correct = {1: 0, 2: 0} # fake, real
    class_total = {1: 0, 2: 0}
    with torch.no_grad():
        for val_images, val_targets in val_loader:
            if len(val_images) == 0:
                continue
            val_images = [img.to(device) for img in val_images]
            val_targets = [{k: v.to(device) for k, v in t.items()} for t in val_targets]
            outputs = model(val_images)
            for output, target in zip(outputs, val_targets):
                if len(output['masks']) > 0 and len(target['masks']) > 0:
                    iou = calculate_iou(
                        output['masks'][:, 0],
                        target['masks'],
                        threshold=0.5
                    )
                    total_iou += iou
                    total_samples += 1
                pred_labels = output['labels'][output['scores'] > 0.5]
                target_labels = target['labels']
                for label in target_labels:
                    label_val = label.item()
                    if label_val in class_total:
                        class_total[label_val] += 1
                    if label_val in pred_labels:
                        class_correct[label_val] += 1
    avg_iou = total_iou / total_samples if total_samples > 0 else 0.0
    class_accuracies = {
        k: class_correct[k] / class_total[k] if class_total[k] > 0 else 0.0
        for k in class_total
    }
    return avg_iou, class_accuracies
```

Appendix 6: Checkpoint Management and Model Persistence

```
def save_checkpoint(model, optimizer, epoch, train_losses, val_losses, train_iou, val_iou,
                    train_class_acc, val_class_acc, checkpoint_path):
    """Save training checkpoint"""
    checkpoint = {
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'train_losses': train_losses,
        'val_losses': val_losses,
        'train_iou': train_iou,
        'val_iou': val_iou,
        'train_class_acc': train_class_acc,
        'val_class_acc': val_class_acc,
    }
    torch.save(checkpoint, checkpoint_path)
    print(f"Checkpoint saved: {checkpoint_path}")
def load_checkpoint(checkpoint_path, model, optimizer):
    """Load training checkpoint"""
    if os.path.exists(checkpoint_path):
        print(f"Loading checkpoint: {checkpoint_path}")
        checkpoint = torch.load(checkpoint_path, map_location=device)
        model.load_state_dict(checkpoint['model_state_dict'])
        optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
        return (checkpoint['epoch'], checkpoint.get('train_losses', []),
                checkpoint.get('val_losses', []), checkpoint.get('train_iou', []),
                checkpoint.get('val_iou', []), checkpoint.get('train_class_acc', []),
                checkpoint.get('val_class_acc', []))
    return 0, [], [], [], [], [], []
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
    torch.save(model.state_dict(), "best_maskrcnn_loss800.pth")
    print(f"Saved best loss model: {best_val_loss:.4f}")
if val_iou > best_iou:
    best_iou = val_iou
    torch.save(model.state_dict(), "best_maskrcnn_iou800.pth")
    print(f"Saved best IoU model: {best_iou:.4f}")
```

Appendix 7: Comprehensive Visualization System

```
def plot_comprehensive_results(train_losses, val_losses, train_iou, val_iou,
                               train_class_acc, val_class_acc):
    """Create comprehensive training plots"""
    epochs = range(1, len(train_losses) + 1)
    fig = plt.figure(figsize=(20, 15))
    plt.subplot(3, 2, 1)
    plt.plot(epochs, train_losses, 'b-o', label='Training Loss', linewidth=2)
    plt.plot(epochs, val_losses, 'r-o', label='Validation Loss', linewidth=2)
    plt.title("Training and Validation Loss", fontsize=14, fontweight='bold')
    plt.xlabel('Epoch')
```

```

plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(3, 2, 2)
plt.plot(epochs, train_iious, 'g-o', label='Training IoU', linewidth=2)
plt.plot(epochs, val_iious, 'orange', marker='o', label='Validation IoU', linewidth=2)
plt.title('Training and Validation IoU', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('IoU Score')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(3, 2, 3)
train_fake_acc = [acc.get(1, 0) for acc in train_class_acc]
val_fake_acc = [acc.get(1, 0) for acc in val_class_acc]
plt.plot(epochs, train_fake_acc, 'purple', marker='o', label='Training Fake Acc', linewidth=2)
plt.plot(epochs, val_fake_acc, 'pink', marker='o', label='Validation Fake Acc', linewidth=2)
plt.title('Fake Class Accuracy', fontsize=14, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(3, 2, 6)
final_metrics = [
    val_losses[-1] if val_losses else 0,
    val_iious[-1] if val_iious else 0,
    val_fake_acc[-1] if val_fake_acc else 0,
    val_real_acc[-1] if val_real_acc else 0
]
metric_names = ['Val Loss', 'Val IoU', 'Fake Acc', 'Real Acc']
colors_bar = ['red', 'orange', 'purple', 'brown']
bars = plt.bar(metric_names, final_metrics, color=colors_bar, alpha=0.7)
plt.title('Final Validation Metrics', fontsize=14, fontweight='bold')
plt.ylabel('Score')
for bar, value in zip(bars, final_metrics):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01,
             f'{value:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.savefig('comprehensive_training_results.png', dpi=300, bbox_inches='tight')
plt.show()

```

Appendix 8: Model Loading and Configuration

```

def get_model(num_classes):
    """Create model with same architecture as training"""
    model = maskrcnn_resnet50_fpn(weights="DEFAULT")
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, 256, num_classes)
    return model

def load_model(model_path, device):
    """Load trained model"""

```

```

model = get_model(num_classes=3) # 3 classes: background, fake, real
model.load_state_dict(torch.load(model_path, map_location=device, weights_only=True))
model.eval()
model.to(device)
return model

# Configuration constants
MODEL_PATH = "C:/Users/bogdan/PycharmProjects/Licence/best_maskrcnn_iou800.pth"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
IMAGE_SIZE = (256, 256)
SCORE_THRESHOLDS = [0.1, 0.3, 0.5, 0.7, 0.9]

```

Appendix 9: Mask-Based Metric Calculations

```

def calculate_iou_masks(pred_mask, gt_mask, threshold=0.5):
    """Calculate IoU between predicted and ground truth masks"""
    pred_binary = (pred_mask > threshold).float()
    gt_binary = gt_mask.float()

    intersection = (pred_binary * gt_binary).sum()
    union = pred_binary.sum() + gt_binary.sum() - intersection

    if union == 0:
        return 1.0 if intersection == 0 else 0.0

    return (intersection / union).item()

def calculate_dice_score(pred_mask, gt_mask, threshold=0.5):
    """Calculate Dice score between predicted and ground truth masks"""
    pred_binary = (pred_mask > threshold).float()
    gt_binary = gt_mask.float()
    intersection = (pred_binary * gt_binary).sum()
    total = pred_binary.sum() + gt_binary.sum()
    if total == 0:
        return 1.0 if intersection == 0 else 0.0
    return (2.0 * intersection / total).item()

def calculate_pixel_accuracy(pred_mask, gt_mask, threshold=0.5):
    """Calculate pixel-wise accuracy"""
    pred_binary = (pred_mask > threshold).float()
    gt_binary = gt_mask.float()
    correct = (pred_binary == gt_binary).float().sum()
    total = gt_binary.numel()
    return (correct / total).item()

```

Appendix 10 : Dataset Evaluation Implementation

```

def evaluate_dataset(self, image_dir, ann_file, dataset_name, max_samples=None):
    """Evaluate model on a dataset"""
    print(f"\n Evaluating {dataset_name} dataset...")
    coco_gt = COCO(ann_file)
    img_ids = list(coco_gt.imgs.keys())

```

```

if max_samples:
    img_ids = img_ids[:max_samples]
all_predictions = []
ground_truths = []
detailed_metrics = []

for img_id in tqdm(img_ids, desc=f"Processing {dataset_name}"):
    try:
        # Load image
        img_info = coco_gt.load_imgs(img_id)[0]
        img_path = os.path.join(image_dir, img_info['file_name'])

        if not os.path.exists(img_path):
            continue

        img = Image.open(img_path).convert("RGB")
        img_resized = img.resize(IMAGE_SIZE)
        tensor = F.to_tensor(img_resized).unsqueeze(0).to(self.device)
        with torch.no_grad():
            outputs = self.model(tensor)[0]
        ann_ids = coco_gt.getAnnIds(imgIds=img_id)
        anns = coco_gt.loadAnns(ann_ids)
        gt_masks, gt_labels, gt_boxes = [], [], []
        for ann in anns:
            if ann.get('iscrowd', 0):
                continue
            mask = coco_gt.annToMask(ann)
            mask = Image.fromarray(mask).resize(IMAGE_SIZE, resample=Image.NEAREST)
            mask = torch.tensor(np.array(mask), dtype=torch.float32)
            if mask.sum() == 0:
                continue
            gt_masks.append(mask)

        for threshold in self.score_thresholds:
            metrics = self._evaluate_single_image(
                outputs, gt_masks, gt_labels, gt_boxes, threshold, img_id
            )
            metrics['dataset'] = dataset_name
            metrics['threshold'] = threshold
            detailed_metrics.append(metrics)

    except Exception as e:
        print(f" Error processing image {img_id}: {str(e)}")
        continue
return detailed_metrics

```

Appendix 11 : Visualization Implementation

```

pythondef plot_evaluation_results(evaluator, save_dir):
    """Create comprehensive evaluation plots"""
    os.makedirs(save_dir, exist_ok=True)
    datasets = list(evaluator.results.keys())
    num_datasets = len(datasets)

    if num_datasets == 0:
        print("No datasets to plot")

```

```

    return
if num_datasets == 1:
    fig, ax = plt.subplots(1, 1, figsize=(8, 6))
    axes = [ax]
elif num_datasets == 2:
    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
else:
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))
    axes = axes.flatten()
for idx, (dataset_name, data) in enumerate(enumerator.results.items()):
    if idx >= len(axes):
        break
    ax = axes[idx]
    summary = evaluator.calculate_summary_metrics(dataset_name)
    thresholds, precisions, recalls = [], [], []
    for threshold in evaluator.score_thresholds:
        key = f'threshold_{threshold}'
        if key in summary:
            thresholds.append(threshold)
            precisions.append(summary[key]['precision'])
            recalls.append(summary[key]['recall'])
    if len(precisions) > 1:
        ax.plot(recalls, precisions, 'b-o', linewidth=2, markersize=6, label='Precision-Recall')
        ax.fill_between(recalls, precisions, alpha=0.3)
    ax.set_xlabel('Recall', fontsize=12)
    ax.set_ylabel('Precision', fontsize=12)
    ax.set_title(f'{dataset_name} - Precision vs Recall', fontsize=14, fontweight='bold')
    ax.grid(True, alpha=0.3)
    ax.legend()
    for i, (r, p, t) in enumerate(zip(recalls, precisions, thresholds)):
        ax.annotate(f'{t:.1f}', (r, p), xytext=(5, 5), textcoords='offset points', fontsize=9)
plt.tight_layout()
plt.savefig(os.path.join(save_dir, 'precision_recall_curves.png'), dpi=300, bbox_inches='tight')
plt.close()

```

Appendix 12: Main Evaluation Function

```

pythondef main():
    """Main evaluation function"""
    print(f" Starting comprehensive evaluation...")
    print(f" Model: {MODEL_PATH}")
    print(f" Device: {DEVICE}")
    print(f" Image size: {IMAGE_SIZE}")
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    os.makedirs(VISUALIZE_DIR, exist_ok=True)
    if not os.path.exists(MODEL_PATH):
        print(f" Model file not found: {MODEL_PATH}")
        return
    print(f" Loading model...")
    model = load_model(MODEL_PATH, DEVICE)
    print(" Model loaded successfully")
    evaluator = DetectionEvaluator(model, DEVICE, SCORE_THRESHOLDS)
    for dataset_name, config in TEST_SETS.items():
        if not os.path.exists(config['image_dir']) or not os.path.exists(config['ann_file']):
            print(f" Skipping {dataset_name} - missing files")
            continue
        evaluator.evaluate_dataset(

```

```

        config['image_dir'],
        config['ann_file'],
        dataset_name,
        max_samples=500
    )
    plot_evaluation_results(evaluator, VISUALIZE_DIR)
    report_path = os.path.join(OUTPUT_DIR, 'evaluation_report.json')
    report = save_evaluation_report(evaluator, report_path)
    print("\n" + "=" * 60)
    print(" 🎉 EVALUATION COMPLETE!")
    print("=" * 60)
    for dataset_name in evaluator.results.keys():
        print(f"\n {dataset_name} Results:")
        best_metrics = report['best_performing_metrics'][dataset_name]
        metrics = best_metrics['metrics_at_best_threshold']
        print(f"  Best threshold: {best_metrics['best_threshold']}")
        print(f"  F1 Score: {metrics.get('f1_score', 0):.3f}")
        print(f"  Precision: {metrics.get('precision', 0):.3f}")
        print(f"  Recall: {metrics.get('recall', 0):.3f}")
        print(f"  IoU: {metrics.get('avg_iou', 0):.3f}")

if __name__ == "__main__":
    main()

```

Appendix 13: DetectionEvaluator class

```

class DetectionEvaluator:
    def __init__(self, model, device, score_thresholds=SCORE_THRESHOLDS):
        self.model = model
        self.device = device
        self.score_thresholds = score_thresholds
        self.results = defaultdict(list)
        self.class_names = {0: 'background', 1: 'fake', 2: 'real'}

    def calculate_summary_metrics(self, dataset_name):
        """Calculate summary metrics for a dataset"""
        summary = {}

        for threshold in self.score_thresholds:
            threshold_metrics = [m for m in detailed_metrics if m['threshold'] == threshold]

            total_tp = sum(m['true_positives'] for m in threshold_metrics)
            total_fp = sum(m['false_positives'] for m in threshold_metrics)
            total_fn = sum(m['false_negatives'] for m in threshold_metrics)

            precision = total_tp / (total_tp + total_fp) if (total_tp + total_fp) > 0 else 0.0
            recall = total_tp / (total_tp + total_fn) if (total_tp + total_fn) > 0 else 0.0
            f1 = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0

```

Appendix 15: Model Comparison Tool Initialization and Configuration

```

class ModelComparisonTool:

```

```

def __init__(self, device='cuda'):
    self.device = torch.device(device if torch.cuda.is_available() else 'cpu')
    self.models = {}
    self.metrics = {}
    self.model_configs = {
        'subset_1000': {
            'name': 'Subset 1K Training',
            'train_samples': 1000,
            'val_samples': 100,
            'color': '#FF6B6B',
            'marker': 'o'
        },
        'subset_2500': {
            'name': 'Subset 2.5K Training',
            'train_samples': 2500,
            'val_samples': 300,
            'color': '#4ECDC4',
            'marker': 's'
        },
        'subset_5000': {
            'name': 'Subset 5K Training',
            'train_samples': 5000,
            'val_samples': 800,
            'color': '#45B7D1',
            'marker': '^'
        }
    }
}

def get_model(self, num_classes=3):
    """Create MaskRCNN model architecture"""
    model = maskrcnn_resnet50_fpn(weights="DEFAULT")
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, 256, num_classes)
    return model

```

Appendix 16: IoU Calculation and Model Evaluation Methods Appendix 14: Summary Metrics Calculation

```

def calculate_iou(self, pred_masks, target_masks, threshold=0.5):
    """Calculate IoU between predicted and target masks"""
    if len(pred_masks) == 0 or len(target_masks) == 0:
        return 0.0
    ious = []
    pred_masks = (pred_masks > threshold).float()
    target_masks = target_masks.float()
    for pred_mask in pred_masks:
        best_iou = 0.0
        for target_mask in target_masks:
            intersection = (pred_mask * target_mask).sum()
            union = pred_mask.sum() + target_mask.sum() - intersection
            if union > 0:
                iou = intersection / union
                best_iou = max(best_iou, iou.item())
        ious.append(best_iou)
    return np.mean(ious) if ious else 0.0

```

```

def evaluate_model_comprehensive(self, model, test_loader, model_name):
    """Comprehensive evaluation of a single model"""
    model.eval()
    metrics = {
        'total_samples': 0,
        'total_iou': 0.0,
        'class_correct': {1: 0, 2: 0}, # fake, real
        'class_total': {1: 0, 2: 0},
        'confidence_scores': [],
        'processing_times': [],
        'detection_counts': []
    }
    print(f" Evaluating {model_name}...")
    with torch.no_grad():
        for batch_idx, (images, targets) in enumerate(test_loader):
            if len(images) == 0:
                continue
            batch_start = time.time()
            images = [img.to(self.device) for img in images]
            targets = [{k: v.to(self.device) for k, v in t.items()} for t in targets]
            outputs = model(images)
            batch_time = time.time() - batch_start
            metrics['processing_times'].append(batch_time)

            for output, target in zip(outputs, targets):
                if len(output['masks']) > 0 and len(target['masks']) > 0:
                    iou = self.calculate_iou(
                        output['masks'][:, 0],
                        target['masks'],
                        threshold=0.5
                    )
                    metrics['total_iou'] += iou
                    metrics['total_samples'] += 1
                high_conf_mask = output['scores'] > 0.5
                pred_labels = output['labels'][high_conf_mask]
                pred_scores = output['scores'][high_conf_mask]
                metrics['detection_counts'].append(len(pred_labels))
                if len(pred_scores) > 0:
                    metrics['confidence_scores'].extend(pred_scores.cpu().numpy())
                target_labels = target['labels']
                for label in target_labels:
                    label_val = label.item()
                    if label_val in metrics['class_total']:
                        metrics['class_total'][label_val] += 1
                    if label_val in pred_labels:
                        metrics['class_correct'][label_val] += 1
    avg_iou = metrics['total_iou'] / metrics['total_samples'] if metrics['total_samples'] > 0 else 0.0
    class_accuracies = {
        k: metrics['class_correct'][k] / metrics['class_total'][k] if metrics['class_total'][k] > 0 else 0.0
        for k in metrics['class_total']
    }
    result = {
        'avg_iou': avg_iou,
        'class_accuracies': class_accuracies,
        'avg_confidence': np.mean(metrics['confidence_scores']) if metrics['confidence_scores'] else 0.0,
        'avg_processing_time': np.mean(metrics['processing_times']),
        'avg_detections_per_image': np.mean(metrics['detection_counts']),
        'total_samples_processed': metrics['total_samples']
    }

```

```

}
return result

```

Appendix 17: Comprehensive Visualization System

```

def create_comparison_plots(self):
    """Create comprehensive comparison plots"""
    if not self.metrics:
        print(" No metrics loaded for comparison!")
        return
    fig = plt.figure(figsize=(24, 18))
    plt.subplot(3, 4, 1)
    for key, config in self.model_configs.items():
        if key in self.metrics:
            history = self.metrics[key].get('training_history', {})
            train_losses = history.get('train_losses', [])
            if train_losses:
                epochs = range(1, len(train_losses) + 1)
                plt.plot(epochs, train_losses, color=config['color'],
                        marker=config['marker'], label=config['name'], linewidth=2)
    plt.title('Training Loss Comparison', fontsize=14, fontweight='bold')
    plt.xlabel('Epoch')
    plt.ylabel('Training Loss')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.subplot(3, 4, 2)
    for key, config in self.model_configs.items():
        if key in self.metrics:
            history = self.metrics[key].get('training_history', {})
            val_losses = history.get('val_losses', [])
            if val_losses:
                epochs = range(1, len(val_losses) + 1)
                plt.plot(epochs, val_losses, color=config['color'],
                        marker=config['marker'], label=config['name'], linewidth=2)
    plt.title('Validation Loss Comparison', fontsize=14, fontweight='bold')
    plt.xlabel('Epoch')
    plt.ylabel('Validation Loss')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.subplot(3, 4, 3)
    for key, config in self.model_configs.items():
        if key in self.metrics:
            history = self.metrics[key].get('training_history', {})
            val_iious = history.get('val_iious', [])
            if val_iious:
                epochs = range(1, len(val_iious) + 1)
                plt.plot(epochs, val_iious, color=config['color'],
                        marker=config['marker'], label=config['name'], linewidth=2)
    plt.title('Validation IoU Comparison', fontsize=14, fontweight='bold')
    plt.xlabel('Epoch')
    plt.ylabel('IoU Score')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('comprehensive_subset_comparison.png', dpi=300, bbox_inches='tight')
    plt.show()

```

Appendix 18: Automated Report Generation System

```
def generate_comparison_report(self):
    """Generate detailed comparison report"""
    if not self.metrics:
        print(" No metrics available for report generation!")
        return
    print("\n" + "=" * 80)
    print(" COMPREHENSIVE SUBSET TRAINING COMPARISON REPORT")
    print("=" * 80)

    # Create comparison table
    comparison_data = []

    for key, config in self.model_configs.items():
        if key in self.metrics:
            metrics = self.metrics[key]
            row = {
                'Model': config['name'],
                'Train Size': config['train_samples'],
                'Val Size': config['val_samples'],
                'Best Val Loss': metrics.get('best_validation_loss', 'N/A'),
                'Best Val IoU': metrics.get('best_validation_iou', 'N/A'),
                'Final Train Loss': metrics.get('final_train_loss', 'N/A'),
                'Final Val Loss': metrics.get('final_val_loss', 'N/A'),
                'Final Val IoU': metrics.get('final_val_iou', 'N/A'),
                'Training Time (hrs)': metrics.get('total_training_time_hours', 'N/A'),
                'Total Epochs': metrics.get('total_epochs', 'N/A')
            }

            # Add class accuracies
            final_class_acc = metrics.get('final_val_class_accuracies', {})
            row['Fake Detection Acc'] = final_class_acc.get(1, 'N/A')
            row['Real Detection Acc'] = final_class_acc.get(2, 'N/A')

            comparison_data.append(row)

    # Create DataFrame and display
    df = pd.DataFrame(comparison_data)
    print("\n PERFORMANCE COMPARISON TABLE:")
    print("-" * 80)
    print(df.to_string(index=False, float_format='%0.4f'))
    if len(comparison_data) > 1:
        # Best performing model identification
        best_iou_idx = df['Best Val IoU'].astype(float).idxmax()
        best_loss_idx = df['Best Val Loss'].astype(float).idxmin()
        print(f" Best IoU Performance: {df.iloc[best_iou_idx]['Model']}")
        print(f" Best Loss Performance: {df.iloc[best_loss_idx]['Model']}")
    report_path = f"subset_comparison_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"
    with open(report_path, 'w') as f:
        json.dump({
            'comparison_data': comparison_data,
            'generation_time': datetime.now().isoformat(),
            'models_compared': list(self.model_configs.keys())
        }, f, indent=4)
    print(f"\n Report saved: {report_path}")
```

References :

- [1] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969.
- [2] T. T. Nguyen *et al.*, ‘Deep Learning for Deepfakes Creation and Detection: A Survey’, *Computer Vision and Image Understanding*, vol. 223, p. 103525, Oct. 2022, doi: 10.1016/j.cviu.2022.103525.
- [3] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia, ‘DeepFakes and Beyond: A Survey of Face Manipulation and Fake Detection’, Jun. 18, 2020, *arXiv*: arXiv:2001.00179. doi: 10.48550/arXiv.2001.00179.
- [4] T.-N. Le, H. H. Nguyen, J. Yamagishi, and I. Echizen, ‘OpenForensics: Multi-Face Forgery Detection And Segmentation In-The-Wild Dataset [V.1.0.0]’. Zenodo, Oct. 13, 2021. Accessed: Jul. 31, 2025. [Online]. Available: <https://zenodo.org/records/5528418>
- [5] B. U. Mahmud and A. Sharmin, “Deep Insights of Deepfake Technology: A Review, , pp. 1-8, 2020, doi: 10.14569/IJACSA.2020.0110572.
- [6] A. Paszke *et al.*, ‘PyTorch: An Imperative Style, High-Performance Deep Learning Library’, Dec. 03, 2019, *arXiv*: arXiv:1912.01703. doi: 10.48550/arXiv.1912.01703.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org>
- [8] T.-Y. Lin *et al.*, ‘Microsoft COCO: Common Objects in Context’, Feb. 21, 2015, *arXiv*: arXiv:1405.0312. doi: 10.48550/arXiv.1405.0312.
- [9] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, ‘Feature Pyramid Networks for Object Detection’, Apr. 19, 2017, *arXiv*: arXiv:1612.03144. doi: 10.48550/arXiv.1612.03144.
- [10] K. He, G. Gkioxari, P. Dollár, and R. Girshick, ‘Mask R-CNN’, Jan. 24, 2018, *arXiv*: arXiv:1703.06870. doi: 10.48550/arXiv.1703.06870.
- [11] W. Song *et al.*, ‘A Quality-Centric Framework for Generic Deepfake Detection’, May 2025, doi: 10.48550/arXiv.2411.05335.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, ‘Deep Residual Learning for Image Recognition’, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.
- [13] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun, ‘OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks’, Feb. 24, 2014, *arXiv*: arXiv:1312.6229. doi: 10.48550/arXiv.1312.6229.
- [14] S. Ren, K. He, R. Girshick, and J. Sun, ‘Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks’, *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: 10.1109/TPAMI.2016.2577031.
- [15] I. J. Goodfellow *et al.*, ‘Generative Adversarial Networks’, Jun. 10, 2014, *arXiv*: arXiv:1406.2661. doi: 10.48550/arXiv.1406.2661.
- [16] R. Caruana, “Multitask Learning,” *Machine Learning*, vol. 28, pp. 41–75, Jul. 1997, doi: 10.1023/A:1007379606734.
- [17] D. P. Kingma and J. Ba, ‘Adam: A Method for Stochastic Optimization’, Jan. 2017, doi: 10.48550/arXiv.1412.6980.
- [18] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. Da Silva, ‘A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit’, *Electronics*, vol. 10, no. 3, p. 279, Jan. 2021, doi: 10.3390/electronics10030279.